

PYTHON-PANDAS-I

STD-XII

INFORMATICS PRACTICES

LESSON-1

2) Data Structures with egs

list → [1,'ab',7.5] heterogeneous data(implicit indexing)

dictionary → {1:'ab',2:'cd'}

Ndarray → [1,'ab',2.5] → ['1','ab',2,5] homogeneous data(implicit indexing)

Series → [1,2,3] homogeneous data(explicit indexing+implicit)

Dataframe → heterogeneous data

	0	1	
0	Mona	20	
1	Gita	30	
2	Mina	40	

0	1	
1	2	
2	3	

1D

2D

1) Order of some data structures learnt so far:-

List

Dictionary

Numpy array → ndarray

Series

Data Frame



3)

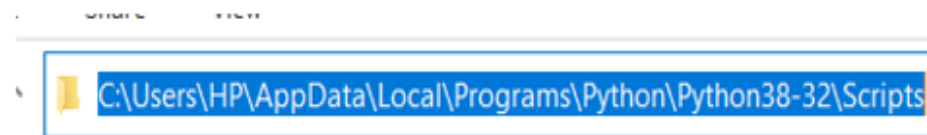
Mutable	Immutable
List	Tuple
Dictionary	Int
Sets	Float
Ndarray(size immutable)	Boolean
Series(size immutable)	String
DataFrames	unicode

1)HOW TO INSTALL THE pandas library in **idle**

- 1)Open idle
- 2)Type

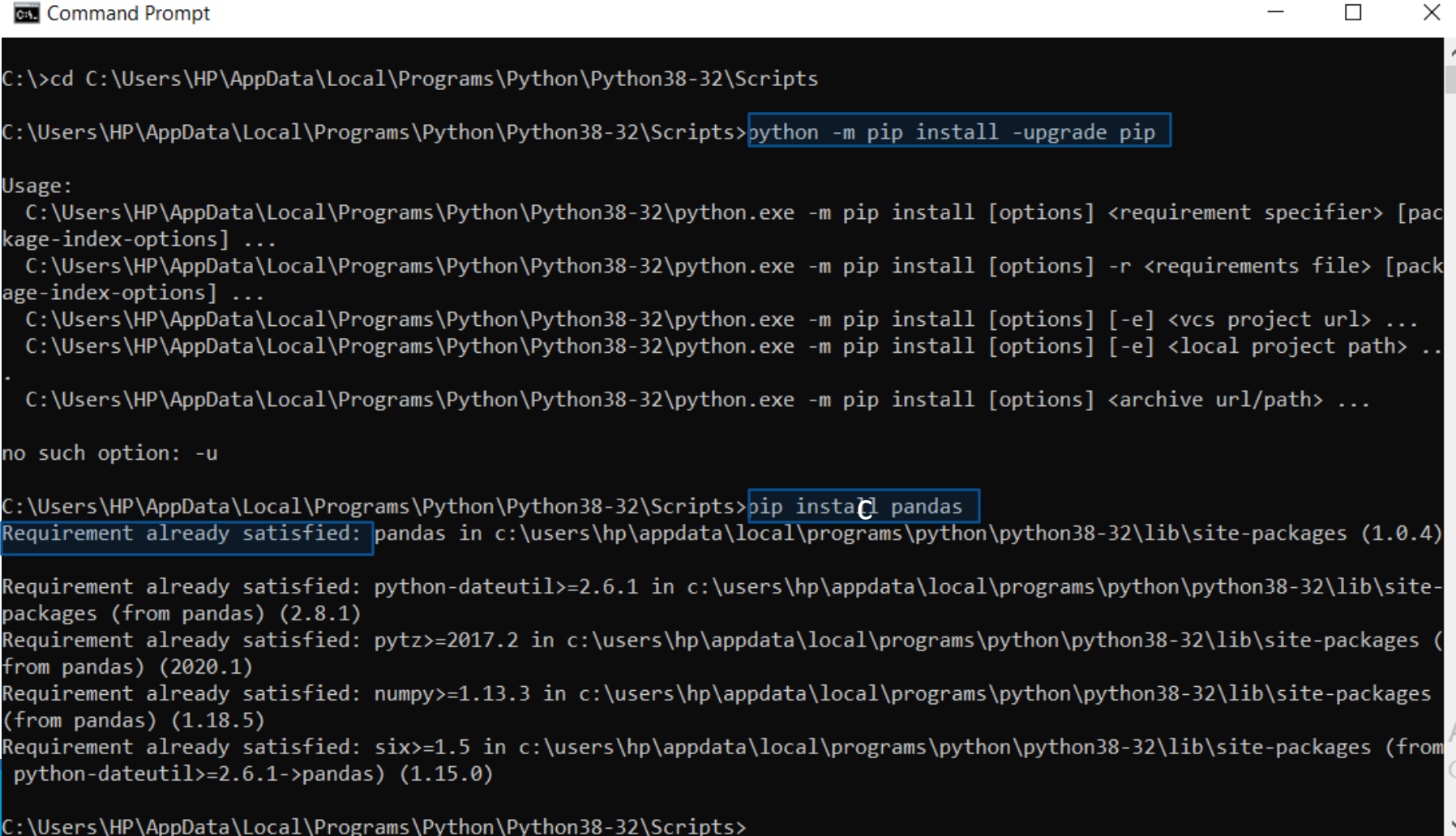
```
>>> import sys
>>> print(sys.executable)
C:\Users\HP\AppData\Local\Programs\Python\Python38-32\pythonw.exe
```
- 3)Select this path till Python38-32,don't select python.exe. Like this

```
>>> import sys
>>> print(sys.executable)
C:\Users\HP\AppData\Local\Programs\Python\Python38-32\pythonw.exe
```
- 4)Paste this path on the address bar of the file explorer window and press enter
- 5) You will see a Scripts folder there. Double click and open that folder of Scripts
- 6)Now go to the address bar of file explorer and copy this path



- 7) open cmd
- 8) type cd\ press enter
- 9) type cd<space>paste the copied path here press enter
- 10) So now you are in the Scripts folder.
- 11) Type **python -m pip install --upgrade pip** (press enter)
- 12) Type **pip install pandas**

**Pip- full form
preferred
installer
program**



```
Command Prompt

C:\>cd C:\Users\HP\AppData\Local\Programs\Python\Python38-32\Scripts

C:\Users\HP\AppData\Local\Programs\Python\Python38-32\Scripts>python -m pip install --upgrade pip

Usage:
  C:\Users\HP\AppData\Local\Programs\Python\Python38-32\python.exe -m pip install [options] <requirement specifier> [package-index-options] ...
  C:\Users\HP\AppData\Local\Programs\Python\Python38-32\python.exe -m pip install [options] -r <requirements file> [package-index-options] ...
  C:\Users\HP\AppData\Local\Programs\Python\Python38-32\python.exe -m pip install [options] [-e] <vcs project url> ...
  C:\Users\HP\AppData\Local\Programs\Python\Python38-32\python.exe -m pip install [options] [-e] <local project path> ...
  C:\Users\HP\AppData\Local\Programs\Python\Python38-32\python.exe -m pip install [options] <archive url/path> ...

no such option: -u

C:\Users\HP\AppData\Local\Programs\Python\Python38-32\Scripts>pip install pandas
Requirement already satisfied: pandas in c:\users\hp\appdata\local\programs\python\python38-32\lib\site-packages (1.0.4)
Requirement already satisfied: python-dateutil>=2.6.1 in c:\users\hp\appdata\local\programs\python\python38-32\lib\site-packages (from pandas) (2.8.1)
Requirement already satisfied: pytz>=2017.2 in c:\users\hp\appdata\local\programs\python\python38-32\lib\site-packages (from pandas) (2020.1)
Requirement already satisfied: numpy>=1.13.3 in c:\users\hp\appdata\local\programs\python\python38-32\lib\site-packages (from pandas) (1.18.5)
Requirement already satisfied: six>=1.5 in c:\users\hp\appdata\local\programs\python\python38-32\lib\site-packages (from python-dateutil>=2.6.1->pandas) (1.15.0)

C:\Users\HP\AppData\Local\Programs\Python\Python38-32\Scripts>
```

I. INTRODUCTION

II. USING PANDAS

III. WHY PANDAS?

IV. PANDAS DATA STRUCTURES

SUMMARY-DATA SERIES()

Syntax `ds=pd.Series(data,[index],[dtype])`

I -Series **did**

II -DataFrame

I- SERIES

A. CREATING SERIES

1) Creating empty series

2) Creating non empty series

i. Python sequence

ii. An ndarray

iii. Python dictionary

iv. Scalar value

3) Creating Series Objects-Additional Functionality

i. Adding NaN in Series

ii. Specify the index and data for the Series

iii. Specify the datatype with index and data

iv. Using a mathematical func. For data of a Series.

B. SERIES OBJECTS ATTRIBUTES

C. ACCESSING A SERIES OBJECTS AND ELEMENTS

i. Access individual elements

ii. Slicing a series objects

D. OPERATIONS ON SERIES OBJECTS

i. Modify elements

ii. head() and tail()

iii. Vector Operations on Series objects

iv. Arithmetic on Series

v. Filtering Entries

vi. Sorting Series Values

E. DIFFERENCE BETWEEN NUMPY AND SERIES OBJECTS

I) INTRODUCTION

extra Pandas has become a very popular choice for data analysis.

Data analysis-refers to the process of evaluating big data sets using analytical and statistical tools so as to discover useful information and conclusions to support business decision-making.

Pandas or Python Pandas-It is a python library for data analysis. Pan|da|s which is a term for quantitative analysis for **multi-dimensional structured data sets**.

panel data system

Def:- Data analysis- It is the process of evaluating big data sets using analytical and statistical tools to discover useful information and conclusions to help in decision making.

The main author of Pandas is **WesMcKinney**

BSD license imposes minimal restrictions on the use and distribution of software.

II) USING PANDAS

Pandas is an **opensource BSD**(Berkeley Software Distribution license) library built for Python Programming Language. It provides high-performance, easy to use data structures and data-analysis tool.

To work with pandas you need to import

import pandas as pd

Extra-

Pandas is a BSD licensed, open source package of Python which is popular for data science. It has been built on the Numpy package. It offers powerful, flexible and expressive data structures that make the manipulation of the data and make the analysis easier. One of the data structures available is the DataFrame. The Pandas DataFrame can be seen as a table. In this data structure, data is organized into rows and columns, which makes a two dimensional data structure. The size of this data structure is mutable and can be modified.

III) WHY PANDAS?

- It can read or write in many diff data formats(integers,float,double,etc)
- It can calculate in all possible ways(across rows and columns)
- It support reshaping the data into diff forms
- It supports visualization using matplotlib and seaborn etc. libraries

more pg(2)

IV) PANDAS DATA STRUCTURES

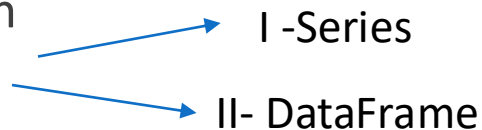
Def:-

DataStructures-They refer to a specialized way of storing data so as to apply a specific type of functionality on them.

A Data structure is a particular way of storing and organizing data in a computer to suit a specific purpose so that it can be accessed and worked with in appropriate ways. ***

Depending on the requirement of the situation a data structure is decided for that situation

Two very basic data structures of Python



There are many more data structures such as panels but we are not studying these right now.

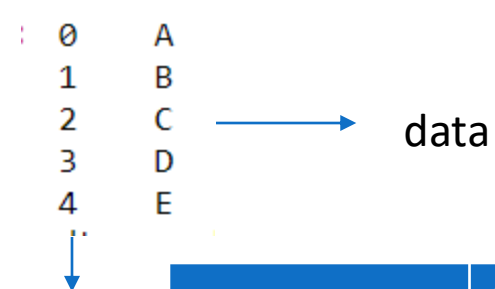
TWO BASIC STRUCTURES SERIES AND DATAFRAME

SERIES-1D data structure of Python Pandas

DataFrame- 2D data structure of Python Pandas

Series

Q Diff between Series and DataFrames



row
index
(index)

```
import pandas as pd
import numpy as np
ds=pd.Series(['A','B','C','D','E'])
ds

0    A
1    B
2    C
3    D
4    E
dtype: object

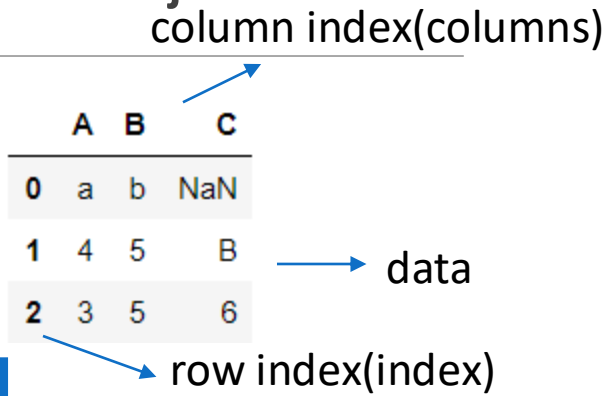
ds[5]='F'
ds

0    A
1    B
2    C
3    D
4    E
5    F
dtype: object
```

	Series
Type Of Data	Homogeneous, 1D
Mutability	Value Mutable (elements value can change)
	Size immutable(once created its size cannot be changed). If you want to add or drop elements, internally a new series object will be created.

DataFrames
Heterogenous,2D
Value Mutable (elements value can change)
Size mutable(once created its size can be changed)You can add or delete elements in an existing df.

DataFrame Object



Object datatype explanation link-
<https://stackoverflow.com/questions/21018654/strings-in-a-dataframe-but-dtype-is-object>
So in Series and df the datatype string is called object datatype(explanation in the link)

```
>>> s1=pd.Series(['a','b','c'])
>>> s1
0    a
1    b
2    c
dtype: object
>>> type(s1[0])
<class 'str'>
```


Data Series(size immutability ka explanation) just for understanding not compulsory(ndarray also size immutable)

```
# doubt about size immutable of ds
import numpy as np
import pandas as pd
ds=pd.Series([1,2,3])
print(ds,id(ds))
ds[3]=4
print(ds,id(ds))
print(ds,id(ds.array))
```

Appending and deleting are allowed, but that doesn't necessarily imply the Series is mutable.

Series/DataFrames are internally represented by NumPy arrays which are immutable (fixed size) to allow a more compact memory representation and better performance.

When you assign to a Series, you're actually calling `Series._setitem_` (which then delegates to `NDFrame.__loc__`) which creates a **new array**. This new array is then assigned back to the same Series (of course, as the end user, you don't get to see this), giving you the illusion of mutability.

```
0    1
1    2
2    3
dtype: int64 17679568
0    1
1    2
2    3
3    4
dtype: int64 17679568
0    1
1    2
2    3
3    4
dtype: int64 17680400
```

Same address of ds

Diff Address of the array
of the ds

EXTRA SLIDE

```
import pandas as pd
test_s = pd.Series([1,2,3])
id(test_s) # output: 140485359734400 (will vary)
len(test_s) # output: 3
test_s[3] = 37
id(test_s) # output: 140485359734400
len(test_s) # output: 4
```

id(test_s.array) reflects this.

same
diff

```
>>> s1=pd.Series([1,2,3])
>>> s1
0    1
1    2
2    3
dtype: int64
>>> id(s1)
130788368
>>> id(s1.array)
60920232
>>> s1[3]=4
>>> s1
0    1
1    2
2    3
3    4
dtype: int64
>>> id(s1)
130788368
>>> id(s1.array)
136304600
```


I) Series-

Def:-

It is a pandas data structure that represents a one dimensional array-like object containing an array of data(of any numpy datatype) and an associated array of data labels, called its index.

*It is 1D

*It has 2 main components

1) an array of actual data

2)an associated array of indexes or data labels.

Index	Data
0	21
1	22
2	23
3	24

Index	Data
Jan	21
Feb	22
March	23
April	24

Index	Data
A	21
B	22
C	23
D	24

A) CREATING SERIES OBJECTS

1) Creating Empty Series Object

Syntax **<series obj>=pandas.Series()** # eg→**ds=pd.Series()**

The above statement will create an empty Series type Objects with no values and having a default datatype as 'float64'

Prog1

```
#prog-1
import numpy as np
import pandas as pd
ds=pd.Series()
print(ds)
```

Series([], dtype: float64)

2) Creating non-Empty Series Objects- To create these objects you need to specify arguments for data and index as the following Syntax:-

<Series Object>=pd.Series(data=[index=idx],[dtype]) did

data- i)Python sequence → list,tuple(set cannot be used here, it is unordered and has no indexing at all)
ii)ndarray
iii)Python Dictionary
iv)scalar value

idx- sequence of numbers or labels of any valid numpy datatypes.

i) Specify data as Python sequence-

Syntax **<Series Obj>=pd.Series(<any python sequence>)**

Prog2

```
>>> s1=pd.Series(data=[1,2,3])
>>> s1
0    1
1    2
2    3
dtype: int64
```

You may or may not write data=

```
#prog2
ds=pd.Series(range(5))
print(ds)
ds1=pd.Series([3.5,5.0,6.5,8.])
ds1
```

```
0    0
1    1
2    2
3    3
4    4
```

dtype: int64

```
0    3.5
1    5.0
2    6.5
3    8.0
dtype: float64
```

```
#eg1
ds=pd.Series('hello')
ds
```

Not like lists **string**

```
0    hello
dtype: object
```

```
>>> l=list("hello")
>>> l
['h', 'e', 'l', 'l', 'o']
```

```
#eg2
ds=pd.Series(['hello','hi'])
ds
```

list

```
0    hello
1      hi
dtype: object
```

```
>>> s1=pd.Series('h','e','l','l','o')
Traceback (most recent call last): error
```

Error
Only 1 seq allowed

```
>>> s1=pd.Series(['h','e','l','l','o'])
>>> s1
0    h
1    e
2    l
3    l
4    o
dtype: object
```

or **list**

```
s1=pd.Series(list("hello"))
s1=pd.Series(['h','e','l','l','o'])
dtype: object
```

Prog3 pg(5)Eg1
Prog4 pg(5)Eg2
Prog5 pg(6)Eg3
Prog6 pg(6)Eg4
Prog7 pg(6)Eg5

22/11/9/0707

EXAMPLE 1 Write code to create a Series object using the Python sequence [4, 6, 8, 10]. Assume that Pandas is imported as alias name *pd*.

SOLUTION

```
import pandas as pd
s1 = pd.Series([4, 6, 8, 10])
print("Series object 1:")
print(s1)
```

list

Output

```
Series object 1:
0    4
1    6
2    8
3   10
dtype: int64
```

EXAMPLE 2 Write code to create a Series object using the Python sequence (11, 21, 31, 41). Assume that Pandas is imported as alias name *pd*.

SOLUTION

```
import pandas as pd
s2 = pd.Series((11, 21, 31, 41))
print("Series object 2:")
print(s2)
```

tuple

Output

```
Series object 2:
0    11
1    21
2    31
3    41
dtype: int64
```

Renaming a series object(extra)
The name of the Series becomes its index or column name, if it is used to form a DataFrame.

```
ss=pd.Series([1,2,3])
print(ss)
print("-----")
a=ss.rename("abc")
print(a)
print("-----")
df=pd.DataFrame(a)
print(df)
print("-----")
df=pd.DataFrame(ss)
print(df)
```

```
0    1
1    2
2    3
dtype: int64
-----
0    1
1    2
2    3
Name: abc, dtype: int64
-----
0    1
1    2
2    3
-----
0    1
1    2
2    3
```

EXAMPLE 3 Write a program to create a Series object using individual characters 'o', 'h', and 'o'. Assume that Pandas is imported as alias name **pd**.

SOLUTION

```
import pandas as pd
s3 = pd.Series(['o', 'h', 'o'])
print("Series object:")
print(s3)
```

list

Output

```
Series object:
0    o
1    h
2    o
dtype: object
```

EXAMPLE 4 Write a program to create a Series object using a string : "So funny". Assume that Pandas is imported as alias name **pd**.

SOLUTION

```
import pandas as pd
print("Series object:")
s4 = pd.Series("So funny")
print(s4)
```

string

Output

```
Series object:
0    So funny
dtype: object
```

EXAMPLE 5 Write a program to create a Series object using three different words : "I", "am", "laughing". Assume that Pandas is imported as alias name **pd**.

SOLUTION

```
import pandas as pd
s5 = pd.Series(["I", "am", "laughing"])
print("Series object:")
print(s5)
```

list

Output

```
Series object:
0      I
1     am
2  laughing
dtype: object
```

```
>>> s5=pd.Series("i am laughing")
>>> s5
0    i am laughing
```

Some ways of creating a numpy array

1) np.array(sequence)

```
>>> import numpy as np
>>> import pandas as pd
>>> arr=np.array([1,2,3])
>>> arr
array([1, 2, 3])
```

array() does not work
with strings
Arr=np.array("hello")
→absurd output

2) np.arange(start,stop,step) → works like range() of python But allows to work with floating numbers

```
>>> arr=np.arange(1,10,1)
>>> arr
array([1, 2, 3, 4, 5, 6, 7, 8, 9])
```

```
import numpy as np
arr=np.arange(1.5,5.5)
arr
array([1.5, 2.5, 3.5, 4.5])
```

3) np.linspace(start,stop,number of elements) returns a float

```
>>> arr=np.linspace(2,10,5)
>>> arr
array([ 2.,  4.,  6.,  8., 10.])
```

float64

4) np.tile([seq],number of times to tile)

```
>>> arr=np.tile([1,2,3],3)
>>> arr
array([1, 2, 3, 1, 2, 3, 1, 2, 3])
```

Numpy array

A numpy array stores homogeneous data in continuous fixed memory locations

For numpy array we need to write →

import numpy as np

Creating Series using numpy arrays-

Egs-

```
>>> s=pd.Series(np.array([1,2,3]))
>>> s
0    1
1    2
2    3
dtype: int32
```

```
>>> s=pd.Series(np.arange(1,5))
>>> s
0    1
1    2
2    3
3    4
dtype: int32
```

```
>>> s=pd.Series(np.linspace(1,5,5))
>>> s
0    1.0
1    2.0
2    3.0
3    4.0
4    5.0
dtype: float64
```

```
>>> s=pd.Series(np.tile([5,6,7],2))
>>> s
0    5
1    6
2    7
3    5
4    6
5    7
dtype: int32
```


ii) Specify data as an ndarray-

Prog-8 :

```
#prog8
nd=np.arange(3,13,3.5)
ds=pd.Series(nd)
print(ds)
```

```
0      3.0
1      6.5
2     10.0
dtype: float64
```

Prog-9 pg(7)Eg6

Prog-10 pg(7)Eg7

```
ds=pd.Series(np.tile([2,3],3))
ds
```

```
0      2
1      3
2      2
3      3
4      2
5      3
dtype: int32
```

Any
sequence
Tuple,set,
Dict

```
>>> n=np.arange(1.0,5.0,2)
>>> n
array([1., 3.])
Skipping 1
array([ 7. ,  9.5, 12. , 14.5, 17. , 19.5, 22. , 24.5])
```

More egs
of
arange→

```
>>> x=np.arange(1,5,0.5)
>>> x
array([1. , 1.5, 2. , 2.5, 3. , 3.5, 4. , 4.5])
>>> x=np.arange(1,5,1.0)
>>> x
array([1., 2., 3., 4.])
>>> x=np.arange(1,5,2.0)
>>> x
array([1., 3.])
>>> x=np.arange(1,5,2.5)
>>> x
array([1. , 3.5])
>>> x=np.arange(1,5,1.5)
>>> x
array([1. , 2.5, 4. ])
>>> x=np.arange(1,6.5,1.5)
>>> x
array([1. , 2.5, 4. , 5.5])
```

EXAMPLE 6 Write a program to create a Series object using an ndarray that has 5 elements in the range 24 to 64.

SOLUTION

```
import pandas as pd
s6 = pd.Series (np.linspace(24, 64, 5))
print(s6)
```

Output

```
0      24.0
1      34.0
2      44.0
3      54.0
4      64.0
dtype: float64
```

EXAMPLE 7 Write a program to create a Series object using an ndarray that is created by tiling a list [3,5], twice).

SOLUTION

```
import pandas as pd
s7 = pd.Series (np.tile([3,5],2))
print(s7)
```

Output

```
0      3
1      5
2      3
3      5
dtype: int32
```

np.tile() works with any sequence

```
>>> s1=pd.Series(np.tile((2,3),3))
>>> s1
0    2
1    3
2    2
3    3
4    2
5    3
dtype: int32
>>> s1=pd.Series(np.tile({4,5},3))
>>> s1
0    {4, 5}
1    {4, 5}
2    {4, 5}
dtype: object
>>> s1
0    {4, 5}
1    {4, 5}
2    {4, 5}
dtype: object
>>> s1=pd.Series(np.tile("hello",3))
>>> s1
0    hello
1    hello
2    hello
dtype: object
>>> s1=pd.Series(np.tile({1:'abc',2:'ppp'},3))
>>> s1
0    {1: 'abc', 2: 'ppp'}
1    {1: 'abc', 2: 'ppp'}
2    {1: 'abc', 2: 'ppp'}
dtype: object
>>> s1[0]
{1: 'abc', 2: 'ppp'}
```

tuple

set

string

dictionary

Extra slide

iii) Specify data as a Python dictionary-

Prog-11

(keys of the dictionary become the index of the series object
and values of the dictionary become the data of the Series object)

```
#prog11  
ds=pd.Series({'Jan':31,'Feb':28,'March':31})  
ds
```

```
Feb      28  
Jan      31  
March    31  
dtype: int64
```

– Indexes are not in the same order as given in the dictionary above

Prog-12 pg(7) Eg-8

EXAMPLE 8 Write a program to create a Series object using a dictionary that stores the number of students in each section of class 12 in your school.

SOLUTION

```
import pandas as pd  
stu = { 'A':39, 'B':41, 'C':42, 'D': 44 }  
s8 = pd.Series(stu)  
print(s8)
```

Output

```
A      39  
B      41  
C      42  
D      44  
dtype: int64
```

We can assign an explicit index to the series object using the index parameter to the Series().
If u assign an **alpha numeric explicit index** then the **default numeric indexing also work**.(0,1,2.....)
If u assign a **numeric explicit index** then the **default numeric indexing does not work**

Eg1-explicit numeric index

```
>>> s=pd.Series([45,55,78],index=[3,4,5])
>>> s
3    45
4    55
5    78
dtype: int64
```

s[0]→error

Eg2-explicit non numeric index

```
>>> s=pd.Series([1,2,3],index=['a','b','c'])
>>> s
a    1
b    2
c    3
dtype: int64
```

s[0]→1

The series can have elements with **same index!!**

Eg3

```
>>> s=pd.Series([4,5,6],['a','b','a'])
>>> s
a    4
b    5
a    6
dtype: int64
>>> s['a']
a    4
a    6
dtype: int64
```

Also
work

```
>>> s[0]
4
>>> s[1]
5
>>> s[2]
6
```

Eg4

```
>>> s=pd.Series([2,3,4],index=[0,1,1])
>>> s
0    2
1    3
1    4
dtype: int64
>>> s[1]
1    3
1    4
dtype: int64
>>> s[2] KeyError: 2
```



This default
numeric indexing
does not work

iv) Specify data as a scalar value:-

A scalar value is one unit of data can be either a number or a chunk of text.

Keep in mind that the length of the data and index should be same, but when you have a scalar or a single value as data, you can have index which is more in length than the data. In that case the data will keep repeating to match the length of the index.

If the data is a python sequence then the index length has to match the length of the data

eg:-

```
import pandas as pd
import numpy as np
ds=pd.Series("hello",index=[1,2,3,4,5])
ds
```

```
1    hello
2    hello
3    hello
4    hello
5    hello
dtype: object
```

Prog13:-

```
#prog13
obj5=pd.Series(10,index=range(0,1)) #or pd.Series(10)
print(obj5)
```

```
0    10
dtype: int64
```

```
obj6=pd.Series(15,index=range(1,6,2))
obj6
```

```
1    15
3    15
5    15
dtype: int64
```

```
obj7=pd.Series('Yet to start',index=['Indore','Delhi','Shimla'])
obj7
```

```
Indore    Yet to start
Delhi     Yet to start
Shimla    Yet to start
dtype: object
```

```
s1=pd.Series(["hello"],index=[1,2,3])
```

Error since ["hello"] is a sequence(list) of length one.

Prog14- pg(8) Eg9

Prog15-pg(8) Eg10

81/9/2020

EXAMPLE 9 Write a program to create a Series object that stores the initial budget allocated (50000/- each) for the four quarters of the year : Qtr1, Qtr2, Qtr3 and Qtr4.

SOLUTION

v.gond

```
import pandas as pd
s9 = pd.Series(50000, index = ['Qtr1', 'Qtr2', 'Qtr3', 'Qtr4'])
print(s9)
```

Output

```
Qtr1    50000
Qtr2    50000
Qtr3    50000
Qtr4    50000
dtype: int64
```

EXAMPLE 10 Total number of medals to be won is 200 in the Inter University games held every alternate year. Write code to create a Series object that stores these medals for games to be held in the decade 2020-2029.

SOLUTION

v.gond

```
import pandas as pd
s10 = pd.Series(200, index = range(2020, 2029, 2))
print(s10)
```

Output

```
2020    200
2022    200
2024    200
2026    200
2028    200
```

3) Creating Series Objects-Additional Functionality

i) Specifying /Adding NaN values in a Series Object

Legal empty value is np.NaN. It is defined in the numpy module and hence you can use np.NaN to specify a missing value **(or use None)**

Note :- None is python internal type which can be considered as an equivalent to Null. It is used to define a null value or no value at all. Prog16 While missing values are NaN in numerical arrays, they are None in object arrays.

```
#prog16
obj3=pd.Series([6.5,np.NaN,2.34])
#obj3=pd.Series([6.5,None,2.34])
obj3
```

```
0    6.50
1     NaN
2    2.34
dtype: float64
```

Eg→

```
>>> obj3=pd.Series([6,None,7])
>>> obj3
0    6.0
1   NaN
2    7.0
dtype: float64
>>> obj3=pd.Series([66,np.NaN,77])
>>> obj3
0    66.0
1   NaN
2    77.0
dtype: float64
```

The datatype
of NaN is
float64

The datatype
of None is
NoneType

ii) Specifying index(es) as well as data with Series()

- While creating Series we can provide the values and also the indexes.
- Both have to be sequences.

Syntax:- <Series Obj>=pandas.Series(data=None, index=None)

Extra-

Extra slide-np.NaN and None

NaN can be used as a numerical value on mathematical operations, while None cannot (or at least shouldn't). None is an internal Python type (NoneType) and would be more like "inexistent" or "empty" than "numerically invalid" in this context.

eg1

```
>>> import numpy as np
>>> l=[1,2,np.NaN]
>>> l
[1, 2, nan]
>>> type(l[2])
<class float>
>>> l=[1,2,None]
>>> l
[1, 2, None]
>>> type(l[2])
<class NoneType>
```

eg3

Arrays and series and dfs allow vectorized operations not lists

```
>>> arr=np.array([1,2,None])
>>> arr*2
Traceback (most recent call last):
  File "<pyshell#18>", line 1, in <module>
    arr*2
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```

error

```
>>> import numpy as np
>>> arr=np.array([1,2,np.NaN])
>>> arr*2
array([ 2.,  4., nan])
```

No error

eg4

```
>>> s=pd.Series([1,2,np.NaN])
>>> s*2
0    2.0
1    4.0
2    NaN
dtype: float64
>>> s=pd.Series([1,2,None])
>>> s*2
0    2.0
1    4.0
2    NaN
dtype: float64
```

No error for Series and df with None and an arithmetic operation

eg2

```
>>> l=[1,2,np.NaN]
>>> l[2]*2
nan
>>> l=[1,2,None]
>>> l[2]*2
Traceback (most recent call last):
  File "<pyshell#27>", line 1, in <module>
    l[2]*2
TypeError: unsupported operand type(s) for *: 'NoneType' and 'int'
```


Prog17

```
#prog17
obj2=pd.Series(data=[32,34,35],index=['A','B','C'])# you can skip the keyword data # u can skip both keywords too
obj2
```

```
A    32
B    34
C    35
dtype: int64
```

Provide the same
number of
indices as values
in data array

```
obj2=pd.Series([32,34,35],['A','B','C'])
obj2
```

```
A    32
B    34
C    35
dtype: int64
```

#using a loop for defining index sequence

```
obj2=pd.Series(range(1,15,3),index=[x for x in 'abcde'])
obj2
```

```
a     1
b     4
c     7
d    10
e    13
dtype: int64
```

error

```
>>> s1=pd.Series(data=[1,2,3],[55,66,77])
SyntaxError: positional argument follows keyword argument
```

```
>>> s1=pd.Series(index=['a','b','c'],[1,2,3])
SyntaxError: positional argument follows keyword argument
```

```
>>> s1=pd.Series(index=['a','b'],data=[1,2])
>>> s1
a     1
b     2
dtype: int64
```

Change order of
index and
data(works)

List comprehension

```
1 s=pd.Series([1,2,3],data[4,5,6])
2 s
```

NameError

EXAMPLE 11 A Python list namely *section* stores the section names ('A', 'B', 'C', 'D') of class 12 in your school. Another list *contri* stores the contribution made by these students to a charity fund endorsed by the school. Write code to create a Series object that stores the contribution amount as the values and the section names as the indexes.

SOLUTION

```
import pandas as pd
section = ['A', 'B', 'C', 'D']
contri = [6700, 5600, 5000, 5200]
s11 = pd.Series(data = contri, index = section)
print(s11)  or  pd.Series(contri, section)
```

Output

```
A    6700
B    5600
C    5000
D    5200
dtype: int64
```

Read

None is the default value for diff parameters taken in case no value is provided for a parameter

iii) Specifying data type along with data and index-

Syntax- `<Series Obj>=pd.Series(data=None, index=None, dtype=None)`

*If u don't specify a datatype, then pandas creates a Series with the nearest datatype to store the given values. You can specify a datatype using Numpy datatype with dtype attribute.

Eg-

```
import pandas as pd
import numpy as np
ds=pd.Series([5.6,3.4],dtype=np.int32)
ds
```

Eg1

```
import pandas as pd
import numpy as np
ds=pd.Series([5.6,3.4])
ds
```

```
0    5.6
1    3.4
dtype: float64
```

ValueError: Trying to coerce float values to integers

Extra slide

```
>>> arr=np.array([1,2,3],dtype=int)
```

Platform specific

```
>>> arr.dtype
```

```
dtype('int32')
```

```
>>> arr=np.array([1,2,3],dtype=int32)
```

Error cause it belongs
to the numpy library

```
Traceback (most recent call last):
```

```
>>> arr=np.array([1,2,3],dtype=np.int32)
```

Platform independant

```
>>> arr.dtype
```

```
dtype('int32')
```

```
>>> arr=np.array([1,2,3],dtype=np.int)
```

Numpy understands
that np.int is same
np.int32

```
>>> arr.dtype
```

```
dtype('int32')
```

int → platform specific

np.int → same as np.int32 (platform independent)

np.int32 → platform independent

Prog19-

```
#prog19
arr=[31,28,31,30]
mon=['jan','feb','march','april']
obj5=pd.Series(data=arr,index=mon,dtype=np.float64)
obj5
```

```
jan      31.0
feb      28.0
march    31.0
april    30.0
dtype: float64
```

```
obj5=pd.Series(data=mon,index=arr,dtype=object)
obj5
```

```
31      jan
28      feb
31    march
30    april
dtype: object
```

iv) Using a mathematical function/expression to create data array in Series()-

The series allows you to define a function or expression that can calculate values for data sequence

Syntax- <Series Obj>=pandas.Series(index=None,data=<function/expression>)

Prog20-

```
#Prog20
```

```
a=np.arange(9,13)
```

Has to be an ndarray

```
print(a)
```

```
obj6=pd.Series(index=a,data=a*2)
```

```
print(obj6)
```

```
obj7=pd.Series(index=a,data=a**2)
```

```
print(obj7)
```

```
[ 9 10 11 12]
```

```
9      18
```

```
10     20
```

```
11     22
```

```
12     24
```

```
dtype: int32
```

```
9      81
```

```
10    100
```

```
11    121
```

```
12    144
```

```
dtype: int32
```

We can do vectorized operation on a numpy array($a*2$ or $a**2$),so that this operation is applied on every element of Numpy array.

But if we apply a similar operation on a python list then the result will be entirely diff.

Lets see....

Prog21-

```
#Prog21  
lst=[9,10,11,12]  
obj=pd.Series(data=(2*lst))  
obj
```

```
0      9  
1     10  
2     11  
3     12  
4      9  
5     10  
6     11  
7     12  
dtype: int64
```

*imp note-while creating a Series object, when u give an index array as a sequence then there is no compulsion for the uniqueness of indexes. i.e you can have duplicate entries in the index array and Python will not raise any error.

Prog22-

```
#prog22
import numpy as np
import pandas as pd
val=np.arange(2.75,50,9.75)
obj3=pd.Series(val,index=['a','b','a','a','b'])
obj3
```

→ No error even if index is same

```
a    2.75
b   12.50
a   22.25
a   32.00
b   41.75
dtype: float64
```

```
print(obj3['a'])
print("-----")
print(obj3[0])
print(obj3[3])
```

```
a    2.75
a   22.25
a   32.00
dtype: float64
-----
2.75
32.0
```

Indices need not be unique in panda series. This will only cause an error if you perform an operation that requires unique indices.

Prog23-pg(12)Eg12

Carefully read the following example. It is similar yet different.

EXAMPLE 12 Sequences `section` and `contri1` store the section names ('A', 'B', 'C', 'D', 'E') and contribution made by them respectively (6700, 5600, 5000, 5200, nil) for a charity. Your school has decided to donate as much contribution as made by each section, i.e., the donation will be doubled.

Write code to create a Series object that stores the contribution amount as the values and the section names as the indexes with datatype as `float32`.

SOLUTION

```
import pandas as pd
import numpy as np
section = ['A', 'B', 'C', 'D', 'E']
```

```
contri1 = np.array([6700, 5600, 5000, 5200, np.NaN])
```

```
s12 = pd.Series(data = contri1 * 2, index = section, dtype = np.float32)
print(s12)
```

This time we have stored the contribution amount in an ndarray `contri1` so that we can double its value through vectorized operation to add school's contribution

This will double each value of the `contri1` ndarray. This is done to reflect the school's contribution to charity.

has to be an array

if put as int?

```
import pandas as pd
import numpy as np
ds = pd.Series([1, 2, 3, np.NaN], dtype=np.int32)
ds
```

ValueError

Trace

ValueError: cannot convert float NaN to integer

Output

```
A    13400.0
B    11200.0
C    10000.0
D    10400.0
E         NaN
dtype: float32
```

Compare the data type of this Series object with the previous object (s11). It is now different from default data type,

IMPORTANT

NOTE

When you store a NaN value in a series object, Pandas requires the data type to be of floating point type. Even if you specify an integer type, Pandas will promote it to a floating point type (automatically) because NaN is not supported by integer types.

Default
dt is
float64

it will
give an
error
If
np.int32
→ error

Series stores values of homogeneous types and still we can store different types of values. How ?



pg13

You have read till now that Series can store homogeneous elements, i.e., elements of the same type. Let us explore this statement little more. Have a look at the following examples :

1. Creating a Series object that stores three integer values :

```
>>> s1 = pd.Series([11, 12, 13])
>>> s1
0    11
1    12
2    13
dtype: int64
```

See, this Series object is storing all integer values and thus, its datatype is one of the integer datatypes

Since all the values being stored are integers, an integer datatype is chosen by Pandas (**int64** here), so that all values can be efficiently stored and processed.

All values are of **int64** types – hence homogeneous datatype.

2. Creating a Series object that stores three floating type values :

```
>>> s2 = pd.Series([11.1, 12.2, 13.3])
>>> s2
0    11.1
1    12.2
2    13.3
dtype: float64
```

See, this Series object is storing all floating-pt. values and thus, its datatype is one of the float datatypes

Since all the values being stored are floating point numbers, a floating-point datatype is chosen by Pandas (**float64** here), so that all values can be efficiently stored and processed.

All values are of **float64** types – hence homogeneous datatype.

3. Creating a Series object that stores a mix of integer and floating type values :

```
>>> s3 = pd.Series([11, 12.2, 13.3])
>>> s3
0    11.0
1    12.2
2    13.3
dtype: float64
```

See, this Series object is storing all numbers – integers and floating-pt. values and thus, its datatype is one of the float datatypes to accommodate all types of numbers

Since all the values being stored are numbers - some integers and some floating point numbers, a floating-point datatype is chosen by Pandas (**float64** here), so that all values can be efficiently stored and processed.

All values are of **float64** types – hence homogeneous datatype.

4. Creating a Series object that stores a mix of different type values :

```
>>> s4 = pd.Series([11, 12.2, 13.3, "Hi"])
>>> s4
0    11
1    12.2
2    13.3
3    Hi
dtype: object
```

See, this Series object is created from values whose datatypes are different. Thus, Pandas will choose a datatype (**object**) which can accommodate all the given values

Since all the values being stored have different datatypes (integers, floating point numbers, string etc.), Pandas will select a datatype, which is capable of holding all these values. Hence, Pandas will choose its datatype as **object**, which is capable of holding any type of value.

All values are of **object** types – hence homogeneous datatype.

B) SERIES OBJECTS ATTRIBUTES



When u create a series object all the information related to it is available through attributes. You can use these attributes in the following format-

Syntax- <Series obj>.<attribute name>

- 1) <Series object>.index returns the index(axis labels) of the Series (sequence)
- 2) <Series object>.values returns an ndarray of the values (array)
- 3) <Series object>.dtype returns the datatype of each element of the series
- 4) <Series object>.shape returns the shape of the series in the form of a tuple(tuple)
- 5) <Series object>.itemsize size in bytes occupied by each element of the series(eg dtype is int 64,then itemsize will return 8)(64bits/8)(the memory occupied by each element of the series object)
- 6) <Series object>.size returns the number of elements in the series object
- 7) <Series object>.nbytes (size*itemsize) returns the total memory occupied by the series object
- 8) <Series object>.hasnans returns True if the series object has any NaN value in it.
- 9) <Series object>.empty returns True if the series object is empty
- 10) <Series object>.count() returns a count of all the non-NaN values in the Series Object
- 11) <Series object>.ndim returns the number of dimensions of the data
- 12) len(<Series object>) returns the total number of elements in the Series Object including NaNs.
- 13) <Series object>.index.name Name of the index; can be used to assign a new name to the index (name of the rowindex in a df)
- 14) <Series object>.name returns or assigns name to series object (name of the col in a df)

(a)Retrieving Index Array and Data Array-

Prog24

```
#prog24
import numpy as np
import pandas as pd
obj5=pd.Series({'Feb':28,'March':31,'April':30})
print(obj5)
print(obj5.index)
print(obj5.values)
print()

obj6=pd.Series(data=np.arange(11,25,3))
print(obj6)
print(obj6.index)
print(obj6.values)
print()

a=np.arange(9,13)
obj7=pd.Series(index=a,data=a*2)
print(obj7)
print(obj7.index)
print(obj7.values)
```

Since index not
specified it will take
range(0 to 4) by
default.

```
Feb      28
March    31
April    30
dtype: int64
Index(['Feb', 'March', 'April'], dtype='object')
[28 31 30]

0      11
1      14
2      17
3      20
4      23
dtype: int32
RangeIndex(start=0, stop=5, step=1)
[11 14 17 20 23]

9      18
10     20
11     22
12     24
dtype: int32
Int64Index([9, 10, 11, 12], dtype='int64')
[18 20 22 24]
```


Name of series object-

Eg-

```
import pandas as pd
import numpy as np
ds=pd.Series([1,2,3])
ds
```

```
0    1
1    2
2    3
dtype: int64
```

```
ds1=pd.Series([11,22,33],name="abc")
ds1
```

```
0    11
1    22
2    33
Name: abc, dtype: int64
```

```
df=pd.DataFrame(ds)
df
```

	0
0	1
1	2
2	3

```
df=pd.DataFrame(ds1)
df
```

	abc
0	11
1	22
2	33



Chapter 1 : PYTHON PANDAS - I

(b) Setting the Index Name

By default Series has no name for its indexes, but you can set its index name by assigning a string to its <Series object>.index.name attribute, e.g.,

```
>>> ser1 = pd.Series([1, 2, 3], index = ['a', 'b', 'c'])
>>> ser1
a    1
b    2
c    3
>>> ser1.index.name = "newInd"
>>> ser1
newInd
a    1
b    2
c    3
```

See, new name for the indexes assigned

Eg-
Index name
of series obj

```
ds=pd.Series([1,2,3])
ds.index.name="aaa"
ds
```

```
aaa
0    1
1    2
2    3
dtype: int64
```

```
df=pd.DataFrame(ds)
print(df)
```

	0
aaa	
0	1
1	2
2	3

(c) Setting Series Name

The <Series>.name attribute can be used to get or set the name of a Series object, e.g.,

```
>>> ser1
a    1
b    2
c    3
dtype: int64
>>> ser1.name = "Mys"
>>> ser1
a    1
b    2
c    3
Name: Mys, dtype: int64
>>> ser1.name
'Mys'
```

Note-The name of the series is like the name of a column when the series adds into a df and vice versa(if a col is extracted from a df its column heading is the name of the series object)

(d) Retrieving datatype(dtype) and itemsize-

For further programs use **obj 2** and **obj3** as follows

```
obj2=pd.Series([3.5,5.,6.5,8.])  
print(obj2)
```

```
0    3.5  
1    5.0  
2    6.5  
3    8.0  
dtype: float64
```

```
obj3=pd.Series([6.5,np.NaN,2.34])  
print(obj3)
```

```
0    6.50  
1     NaN  
2    2.34  
dtype: float64
```

To know the datatype of each element of the series we use<obj name>.dtype

To know the number of bytes allocated to each element of the object we use<obj name>.itemsize

To know the type of object itself we use the type() method of python.

Prog25-

```
#prog25  
print(obj2.dtype)  
#print(obj2.itemsize) —————> Deprecated  
print(type(obj2))  
|
```

```
float64  
<class 'pandas.core.series.Series'>
```

(e) Retrieving shape(including NaNs)

The shape tells us how many elements it contains including missing or empty values. Since there is only one axis in Series is shown as (<n>,) where n is the number of elements in the object eg:-

Prog-26

```
print(obj2.shape)
print(obj3.shape)
```

```
(4,)
(3,)
```

```
obj2=pd.Series([3.5,5.,6.5,8.])
print(obj2)
```

```
0    3.5
1    5.0
2    6.5
3    8.0
dtype: float64
```

```
obj3=pd.Series([6.5,np.NaN,2.34])
print(obj3)
```

```
0    6.50
1     NaN
2    2.34
dtype: float64
```

*Series are always 1D.

(f) Retrieving dimensions, size and nbytes

Prog-27

```
#prog27
print(obj2.size,obj3.size)
print(obj2.nbytes,obj3.nbytes)
print(obj2.ndim)
```

```
4 3
```

```
32 24
```

```
1
```

Size x itemsize (4x8)=32 (3x8)=24

(g) Checking for emptiness and Presence of NaNs

Empty-It means, any of the axes are of length 0.If the data series has NaN or None it is not considered empty.

Prog28

```
#prog29
import pandas as pd
import numpy as np
obj1=pd.Series()
print(obj1.empty)
print(obj2.empty)
print(obj3.empty)
```

True
False
False

Prog29:

```
ds=pd.Series([np.NaN,np.NaN])
print(ds)
print(ds.empty)
```

0 NaN
1 NaN
dtype: float64
False

```
obj2=pd.Series([3.5,5.,6.5,8.])
print(obj2)
```

0 3.5
1 5.0
2 6.5
3 8.0
dtype: float64

```
obj3=pd.Series([6.5,np.NaN,2.34])
print(obj3)
```

0 6.50
1 NaN
2 2.34
dtype: float64

*if you want to check if the series object has NaN, you can use len() to get the total number of elements and<series>.count() to get the count of non-NaN values in series object.

Prog-30

```
#Prog-30
print(obj2.hasnans)
print(obj3.hasnans)
```

False
True

Prog-31

```
#prog-31
ds=pd.Series([2,np.NaN,7])
print(ds.count())
print(len(ds))
```

2
3



Prog32-pg(15)Eg13 pg(17)

```
ds=pd.Series([1000,2000,23000,1200],index=['A','B','C','D'])
print(ds)
print()
```

```
A    1000
B     2000
C   23000
D     1200
dtype: int64
```

```
ds1=pd.Series([67000,2000,3000,23000],index=['A','B','C','D'],dtype=np.float32)
print(ds1)
```

```
A    67000.0
B     2000.0
C     3000.0
D    23000.0
dtype: float32
```

```
print(ds.dtype,"\t",ds1.dtype)
print(ds.shape,"\t",ds1.shape)
print(ds.nbytes,"\t",ds1.nbytes)
print(ds.ndim,"\t",ds1.ndim)
#print(ds.itemsize,"\t",ds1.itemsize)
print(ds.hasnans,"\t",ds1.hasnans)
print(ds.empty,"\t",ds1.empty)
```

```
int64    float32
(4,)      (4,)
32        16 8x4=32 4x4=16
1          1
False     False
False     False
```

C) ACCESSING A SERIES OBJECT AND ITS ELEMENTS

i) Accessing individual elements:-

To access individual elements you can give its index in square [] along with its name.

Syntax- <Series obj>[valid index]

Prog 33-

```
import pandas as pd
import numpy as np
ds=pd.Series(np.arange(10,60,10))
print(ds)
```

```
0    10
1    20
2    30
3    40
4    50
dtype: int32
```

```
print(ds[3])
```

```
40
```

```
print(ds[2])
```

```
30
```

```
ds1=pd.Series(np.arange(10,60,10),index=['a','b','c','a','a'])
print(ds1)
```

```
a    10
b    20
c    30
a    40
a    50
dtype: int32
```

- Access individual elements RT→element
- Slicing a series objects RT→ series object

```
print(ds1['a'])
```

```
a    10
a    40
a    50
dtype: int32
```

```
print(ds1['x'])
```

KeyError: 'x'

Negative indexing
works with
alphanumeric indexes
only

Eg1- negative indexing in pandas

```
import pandas as pd
import numpy as np
ds=pd.Series([1,2,3],['a','b','c'])
ds[-1]
```

```
3
```

eg2

```
import pandas as pd
import numpy as np
ds=pd.Series([1,2,3])
ds[-1]
```

KeyError

ii) Extracting Slices-

If u use numeric values in slicing, then it is treated as position which goes till stop-1
If u use alphanumeric labels while slicing, then it includes the stop element.

***imp-Slicing takes place position wise and not the index wise in a series object**

Internally there is a position associated with element-first element gets position as 0, second element gets position as 1 and so on. (Irrespective of their index labels, position always begins from 0)

A slice object is created from a Series Obj using the syntax-

syntax :-<object>[start:end:step]

but the start and end signify the position of the elements not their indexes.

The output of the slice is a series object.

```
>>> s=pd.Series([1,2,3],['a','b','c'])
>>> s
a    1
b    2
c    3
dtype: int64
>>> s['a':'c']
a    1
b    2
c    3
dtype: int64
>>> s[0:2]
a    1
b    2
dtype: int64
```

```
obj5=pd.Series([28,31,31],index=['Feb','Jan','Mar'])
obj5
```

0	Feb	28
1	Jan	31
2	Mar	31

dtype: int64

obj5[1:]

	Jan	31
	Mar	31

dtype: int64

```
obj6=pd.Series(np.arange(11,24,3))
obj6
```

0	11
1	14
2	17
3	20
4	23

dtype: int32

obj6[2:5]

2	17
3	20
4	23

dtype: int32

```
obj7=pd.Series(np.arange(18,25,2),index=range(9,13))
obj7
```

0	9	18
1	10	20
2	11	22
3	12	24

dtype: int32

obj7[0::2]

	9	18
	11	22

dtype: int32

obj7[10:15]

Series([], dtype: int32)

obj7[:: -1]

	12	24
	11	22
	10	20
	9	18

dtype: int32

obj7[:2]

	9	18
	10	20

dtype: int32

Slices created, are views of the data series, so any change in the slice or main dataset will reflect in both the places- (shallow copy)

Prog36- Eg14 pg(20)

Prog35-

series

```
#prog35
ds=pd.Series([1,2,3,4,5,6])
dd=ds[:2]
print(dd)
```

```
0    1
1    2
dtype: int64
```

```
dd[0]=10
print(dd)
```

```
0    10
1     2
dtype: int64
```

```
print(ds)
```

```
0    10
1     2
2     3
3     4
4     5
5     6
dtype: int64
```

```
ds[0]=1
print(dd)
```

```
0    1
1     2
dtype: int64
```

EXAMPLE 14 Consider a Series object *s8* that stores the number of students in each section of class 12 (as shown below).

A	39
B	41
C	42
D	44

First two sections have been given a task of selling tickets @ 100/- per ticket as part of a social experiment. Write code to display how much they have collected.

SOLUTION

```
import pandas as pd
print("Tickets amount:")
print(s8[:2]*100)
```

or

`s8['A':'B']*100`

Output

```
Tickets amount:
A    3900
B    4100
dtype: int64
```

Reminder-

While slicing a list, any change in one will not reflect in the other .
So slicing a list creates a True copy of the list.
List slicing → True copy (list.copy(), list(), l[:]) 3 ways of true copy
Series slicing → shallow copy

D) OPERATIONS ON SERIES OBJECT

Prog37

i) Modifying elements of Series Object

Syntax:- <Series Object>[<index>]=<new data values>1 item
<Series Object>[start:stop]=<new data value>1 element
or exact no.of elements as the left side

Eg:-

```
#Eg
ds=pd.Series([1,2,3,4,5],index=['a','b','c','d','e'])
print(ds)
print()
ds[1:5:2]=380
print(ds)
```

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

```
a    1
b   380
c     3
d   380
e     5
dtype: int64
```

```
#Prog37
import pandas as pd
import numpy as np
obj1=pd.Series([1,2,3,4,5])
obj1[0]=10
print(obj1)
```

```
0    10
1     2
2     3
3     4
4     5
dtype: int64
```

```
obj1[2:4]=-15.75      #not allowed in lists
print(obj1)
```

```
0    10.00
1     2.00
2   -15.75
3   -15.75
4     5.00
dtype: float64
```

have become float implicitly

```
obj1[2:4]=(10,20)
print(obj1)
```

```
0    10.0
1     2.0
2    10.0
3    20.0
4     5.0
dtype: float64
```

EXAMPLE 15 Consider the Series object `s13` that stores the contribution of each section, as shown below :

0	A	6700
1	B	5600
2	C	5000
3	D	5200

Output

Series object after modifying amounts:
A 7600
B 5600
C 7000
D 7000

Write code to modify the amount of section 'A' as 7600 and for sections 'C' and 'D' as 7000. Print the changed object.

SOLUTION

```
import pandas as pd
s13[0] = 7600 OR s13['A'] = 7600 # to modify section 'A''s amount
s13[2:] = 7000 # to modify sections 'C' and 'D''s amount
print("Series object after modifying amounts:")
print(s13)
```

~~`s13['C':]` x slicing is based on position not indexing~~

It is
working

or

`s13[2:4]???`

Renaming Indexing-

You can change indexes of Series object by assigning new index array to its index attribute

Syntax <Object>.index=<new index array>

Prog39

```
#prog39
ds=pd.Series([1,2,3,4,5])
print(ds)
print()
ds.index=['a','e','i','o','u']
print(ds)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64
```

```
a    1
e    2
i    3
o    4
u    5
dtype: int64
```

****Ensure that the size of the new index matches with the existing index array size.**

****remember that series are value mutable but size immutable.**

ii) head() and tail() functions-

head(<n>)-used to fetch first n rows from pandas object.

tail(<n>)- returns the last n rows from pandas object.

Syntax <pandas object>.head(<n>)
 <pandas object>.tail(<n>)

If you don't provide the n parameter then head will return the first 5 and tail will return the last 5 rows from the series object.

Prog40-

```
#prog39
ds=pd.Series(np.arange(1,20))
print(ds.head())
print(ds.tail())
print(ds.head(3))
print(ds.tail(3))
```

```
0      1
1      2
2      3
3      4
4      5
dtype: int32
14     15
15     16
16     17
17     18
18     19
dtype: int32
0      1
1      2
2      3
dtype: int32
16     17
17     18
18     19
dtype: int32
```

Prog41-pg(20)Eg16 pg(22)



EXAMPLE 16 A Series object *trdata* consists of around 2500 rows of data. Write a program to print the following details :

(i) First 100 rows of data (ii) Last 5 rows of data

SOLUTION

```
import pandas as pd
:           # trdata object's creation or loading happens here
print(trdata.head(100))
print(trdata.tail())
```

default is 5

iii)Vector Operations on Series Objects-

Vector operations mean that if you apply a function or expression then it is individually applied on each item of the object.

-Series objects are built upon Numpy arrays(ndarrays),they also support vectorized operations just like ndarrays.

Prog42-

```
#prog42
ds=pd.Series([1,2,3,16])
print(ds*2)
```

```
0      2
1      4
2      6
3     32
dtype: int64
```

```
print(ds+2)
```

```
0      3
1      4
2      5
3     18
dtype: int64
```

```
print(ds**2)
```

```
0      1
1      4
2      9
3    256
dtype: int64
```

```
print(ds>15)
```

```
0    False
1    False
2    False
3     True
dtype: bool
```

Original series will not change .

Don't
copy
extra eg

```
import pandas as pd
import numpy as np
ds=pd.Series([1,2,'a','b',None])
ds*2
```

```
0      2
1      4
2     aa
3     bb
4     NaN
dtype: object
```

iv) Arithmetic on Series objects

ob1+ob3		ob1*ob3		ob1/ob3	
0	3.0	0	2.0	0	0.5
1	3.0	1	2.0	1	2.0
2	6.0	2	9.0	2	1.0
3	8.0	3	16.0	3	1.0
4	NaN	4	NaN	4	NaN

ob1		ob2		ob3		ob4		ob5	
0	1	a	2	0	2	0	3	a	1
1	2	b	1	1	1	1	4	b	2
2	3	c	3	2	3	2	5	c	3
3	4	d	4	3	4	4	7	d	1
		e	3	4	1	5	8	e	14
						6	9		
						7	10		

ob2+ob5		ob1+ob4		ob1+ob2	
a	3	0	4.0	0	NaN
b	3	1	6.0	1	NaN
c	6	2	8.0	2	NaN
d	5	3	10.0	3	NaN
e	17	4	NaN	a	NaN
		5	NaN	b	NaN
		6	NaN	c	NaN
		7	NaN	d	NaN
				e	NaN

When you perform arithmetic operations on two series type objects the data is aligned on the basis of matching index(called **Data Alignment in pandas objects**) and then perform arithmetic operations. For non overlapping indexes, the arithmetic operations results as a NaN(Not a Number)

new-Pg(24)

Prog43 Eg17 pg(25)

Prog44 Eg18 pg(25)

EXAMPLE 17 Number of students in classes 11 and 12 in three streams ('Science', 'Commerce' and 'Humanities') are stored in two Series objects c11 and c12. Write code to find total number of students in classes 11 and 12, stream wise.

SOLUTION

```
import pandas as pd
# creating Series objects
c11 = pd.Series(data = [30, 40, 50], index = ['Science', 'Commerce', 'Humanities'])
c12 = pd.Series(data = [37, 44, 45], index = ['Science', 'Commerce', 'Humanities'])
# adding two objects to get total no. of students
print("Total no. of students")
print(c11+c12) # series objects arithmetic
```

Output

Total no. of students	
Science	67
Commerce	84
Humanities	95
dtype: int64	

EXAMPLE 18 Object1 **Population** stores the details of population in four metro cities of India and Object2 **AvgIncome** stores the total average income reported in previous year in each of these metros. Calculate income per capita for each of these metro cities.

SOLUTION

```
import pandas as pd
Population = pd.Series([10927986, 12691836, 4631392, 4328063 ], \
index = ['Delhi', 'Mumbai', 'Kolkata', 'Chennai'])
AvgIncome = pd.Series([72167810927986, 85087812691836, 4226784631392, 5261784328063 ], \
index = ['Delhi', 'Mumbai', 'Kolkata', 'Chennai'])
perCapita = AvgIncome / Population ← Expression to calculate per capita income
print("Population in four metro cities ")
print(Population)
print("Avg. Income in four metro cities")
print(AvgIncome)
print("Per Capita Income in four metro cities ")
print(perCapita)
```

Statement continuation mark. Do not type it while typing code in a .py file, rather type the whole statement in single line

The output produced by above file is as shown on the right.

See how easy it has become to calculate per capita income if we have huge data stored about all the cities and average income etc.

```
In [113]: runfile('E:/Informatics Practi
perCapita.py', wdir='E:/Informatics Prac
Population in four metro cities
Delhi      10927986
Mumbai     12691836
Kolkata     4631392
Chennai     4328063
dtype: int64
Avg. Income in four metro cities
Delhi      72167810927986
Mumbai     85087812691836
Kolkata     4226784631392
Chennai     5261784328063
dtype: int64
Per Capita Income in four metro cities
Delhi      6.603944e+06
Mumbai     6.704137e+06
Kolkata     9.126321e+05
Chennai     1.215757e+06
dtype: float64
```

2020/6/23 22:54

v) Filtering entries- You can filter out entries from a Series objects using expressions that are of Boolean type.

```
ob1
0    1
1   20
2   10
3    4
dtype: int64
```

```
ob2
a    20
b     1
c     3
d   40
e     3
dtype: int64
```

When you apply a comparison operator directly on a pandas Series object, then it works like a vectorized operation and applies this check on each individual element of Series object and returns Boolean.

Prog45-

```
ob1>5
0    False
1     True
2     True
3    False
dtype: bool
```

```
ob1[ob1>5]
1    20
2    10
dtype: int64
```

When you apply this check with the Series Object inside [], you will find that it returns filtered result containing **only the values** that return True.

```
ob2>10
a     True
b    False
c    False
d     True
e    False
dtype: bool
```

```
ob2[ob2>10]
a    20
d    40
dtype: int64
```

Prog46-Eg19 pg(26)

Prog 47-Eg20 pg(27)

EXAMPLE 20 Series object s11 stores the charity contribution made by each section (see below) :

A	6700
B	5600
C	5000
D	5200

Write a program to display which sections made a contribution more than ₹5500/-

SOLUTION

```
import pandas as pd
:           # s11 created or loaded here
print("Contribution > 5500 by :")
print(s11[s11 > 5500])
```

Output

```
Contribution > 5500 by :
A    6700
B    5600
dtype: int64
```

EXAMPLE 19 What will be the output produced by the following program ?

```
import pandas as pd
info = pd.Series(data = [31, 41, 51])
print(info)
print(info > 40)
print(info[info > 40])
```

SOLUTION

```
0    31
1    41
2    51
dtype: int64
0    False
1     True
2     True
dtype: bool
1    41
2    51
dtype: int64
```

← Series object

← Vectorized operation result

← Filtered result

vi)Sorting Series Values- Temporary change

You can sort the values of a Series Object on the basis of values and indexes

Sorting on the basis of Values

Syntax- <Series Obj>.sort_values([ascending=True/False]) default-True

Prog48-

```
#prog48
ds=pd.Series([1000,6700,2300])
print(ds)
ds=ds.sort_values()
print(ds)
ds=ds.sort_values(ascending=False)
print(ds)
```

Or ds.sort_values(inplace=True)

```
0    1000
1    6700
2    2300
dtype: int64
0    1000
2    2300
1    6700
dtype: int64
1    6700
2    2300
0    1000
dtype: int64
```


Sorting on the basis of Index-(temporary change)



Syntax- <Series obj>.sort_index([ascending=True/False) default-True

Prog49-

```
print(ds)
```

```
ds=ds.sort_index(ascending=False)
```

Or ds.sort_index(ascending=False,inplace=True)

```
print(ds)
```

```
ds=ds.sort_index()
```

```
print(ds)
```

Original
ds→

```
2    2300
1    6700
0    1000
dtype: int64
2    2300
1    6700
0    1000
dtype: int64
0    1000
1    6700
2    2300
dtype: int64
```

E) DIFFERENCE NUMPY ARRAYS AND SERIES OBJECTS-

NDARRAYS

1) We can perform arithmetic operations on arrays only if the shape of the 2 arrays match, else we get an error.

2) The indexes are always numeric starting from 0. (Implicit indexing)

```
#eg
import numpy as np
arr=np.array([1,2,3,4])
arr1=np.array([5,6,7])
print(arr+arr1)
```

ValueError: operands could not be broadcast together with shapes (4,) (3,)

3) Numpy arrays cannot have the same index values for 2 items

4) Ndarrays can have any dimensions

SERIES OBJECT

1) With Series objects, the data of the 2 Series Objects is aligned as per matching indexes and operations are performed on them and for non matching indexes NaN is returned.

2) Series objects can have any type of indexes , including numbers(not necessarily starting from zero), letters, labels strings etc.(explicit indexing is possible)

```
#eg
import pandas as pd
ds=pd.Series([1,2,3])
ds1=pd.Series([3,4,5,6])
print(ds+ds1)
```

```
0    4.0
1    6.0
2    8.0
3    NaN
dtype: float64
```

3) Series can have the same labels for 2 items.

4) Series has only one dimension

1.7.1 Series Objects vs. Lists

Series objects are essentially 1D data structures while lists can be 1D or even 2D (when nested). Following table explores Series objects *vs.* lists.

Table 1.3 Series Objects vs. Lists

S.No.	Series Object	Lists
1.	It is essentially a 1D data structure.	It can be 1D and even multi-dimensional with nested lists in it.
2.	It can have numeric indexes as well as labels.	It can take numeric indexes only.
3.	It supports explicit indexing, <i>i.e.</i> , we can programmatically choose, provide and change indexes in terms of numbers or labels.	It does not support explicit indexing ; only supports implicit indexing whereby the indexes are implicitly given 0 onwards in forward indexing and -1 onwards in backward indexing.
4.	Indexes can be duplicate.	Indexes cannot be duplicate.
5.	Homogeneous elements : Series objects store elements of same data type (values may be different but their datatype is the same for each element)	Heterogeneous elements : Lists can store elements of different data types.

Pg 29→learn the entire page from the TB properly



```
dict={"a":{"abc":[1,2,3]}, "b":{"def":[4,5,6]}}  
dict['a']['abc']
```

[1, 2, 3] 2 D(dictionary)



Table 1.4 Series Objects vs. Dictionaries		
S.No.	Series Object	Dictionary
1.	It is essentially a 1D data structure.	It can be 1D and even multi-dimensional with nested dictionaries in it as values.
2.	It can be thought of as similar to dictionaries as dictionaries store values against keys, Series stores values against indexes/labels.	It can be thought of similar to a series object such that its <i>keys</i> can be considered equivalent to that of indexes/labels of Series object and <i>values</i> equivalent to the elements stored in Series object.
3.	Its indexes can be numbers or labels only.	Its keys can only be of immutable types only.

Series Object vs 1D Numpy
Array→pg29 (pl learn)

SOME ADDITIONAL OPERATIONS ON SERIES OBJECTS-

1)RE-INDEXING-(temporary change)(inplace does not work)

If you need to create a similar object with a different order of same indexes.

Syntax- `<Series Object>=<Object>.reindex(<sequence with new order of the indexes>)`

In this , the same data values and their indexes will be stored in the new object as per the defined order of index in the reindex().

Prog-50

```
#prog50
a=np.array(range(10,15))
s=pd.Series(a)
print(s)
s=s.reindex([4,3,1,2,0])
print(s)
```

```
0    10
1    11
2    12
3    13
4    14
dtype: int32
4    14
3    13
1    11
2    12
0    10
dtype: int32
```

Note-If the reindex consists of a new index , we get a NaN for that index but not an error.

```
ds=pd.Series([1,2,3])
print(ds)
ds=ds.reindex([2,1,4])
ds
0    1
1    2
2    3
dtype: int64

2    3.0
1    2.0
4    NaN
dtype: float64
```

2) Dropping Entries from an axis-(temporary)

To remove an entry from a series Object we can use drop()

Syntax `<Series Object>.drop(<row label>)` What ever u can see removing 1 element

`<Series Object>.drop(<list of row labels>)` What ever u can see removing more than 1 element at one go

Prog-51

```
#prog51
ds=pd.Series([1,2,3,4,5])
print(ds)
ds=ds.drop(1) or ds.drop(1,inplace=True)
print(ds)
```

```
0    1
1    2
2    3
3    4
4    5
dtype: int64

0    1
2    3
3    4
4    5
dtype: int64
```

END OF SERIES

X

X

Prog-52

```
ds=pd.Series([1,2,3,4,5])
print(ds)
ds=ds.drop([1,3,4])
ds

0    1
1    2
2    3
3    4
4    5
dtype: int64

0    1
2    3
dtype: int64
```

You can also use del keyword of python to delete an object-

Eg
Del does not allow To delete multiple Elements at one go From a series object.
** del works with list slicing and Individual elements

```
s1=pd.Series([1,2,3],('a','b','c'))
s1

a    1
b    2
c    3
dtype: int64

del s1['a']

s1

b    2
c    3
dtype: int64
```


Multiple Choice Questions

1. To create an empty Series object, you can use :

- (a) `pd.Series(empty)`
- (b) `pd.Series(np.NaN)`
- (c) `pd.Series( )`
- (d) all of these

2. To specify datatype `int16` for a Series object, you can write :

- (a) `pd.Series(data = array, dtype = int16)`
- (b) `pd.Series(data = array, dtype = numpy.int16)`
- (c) `pd.Series(data = array.dtype = pandas.int16)`
- (d) all of the above

3. To get the number of dimensions of a Series object, _____ attribute is displayed.

- (a) `index`
- (b) `size`
- (c) `itemsize`
- (d) `ndim`

4. To get the size of the datatype of the items in Series object, you can display _____ attribute.

- (a) `index`
- (b) `size`
- (c) `itemsize`
- (d) `ndim`

5. To get the number of elements in a Series object, _____ attribute may be used.

- (a) `index`
- (b) `size`
- (c) `itemsize`
- (d) `ndim`

6. To get the number of bytes of the Series data, _____ attribute is displayed.

- (a) `hasnans`
- (b) `nbytes`
- (c) `ndim`
- (d) `dtype`

7. To check if the Series object contains NaN values, _____ attribute is displayed.

- (a) `hasnans`
- (b) `nbytes`
- (c) `ndim`
- (d) `dtype`

Chapter 1

8. To display third element of a Series object `S`, you will write _____ .

- (a) `S[:3]`
- (b) `S[2]`
- (c) `S[3]`
- (d) `S[:2]`

9. To display first three elements of a Series object `S`, you may write _____ .

- (a) `S[:3]`
- (b) `S[3]`
- (c) `S[3rd]`
- (d) all of these

10. To display last five rows of a Series object `S`, you may write _____ .

- (a) `head( )`
- (b) `head(5)`
- (c) `tail( )`
- (d) `tail(5)`

11. Missing data in Pandas object is represented through :

- (a) `Null`
- (b) `None`
- (c) `Missing`
- (d) `NaN`

DATA SERIES TB pg(56)

pg(63)

Fill in the Blanks

1. _____ is a popular data-science library of Python.
2. A _____ is a Pandas data structure that represents a 1 D array like object.
3. A _____ is a Pandas data structure that represents a 2 D array like object.
4. You can use numpy._____ for missing data.
5. To specify datatype for a Series object, _____ argument is used.
6. The _____ function on Series object returns total elements in it including NaNs.
7. The _____ function on Series object returns only the count of non-NaN values in it.
8. Series is _____ mutable.
9. Series is not _____ mutable.
10. Dataframe is _____ mutable as well as _____ mutable.

DATA SERIES

True/False Questions

1. A Pandas Series object can be thought of as a column or a row, essentially.
2. Both Series and DataFrame are one dimensional data structure objects.
3. While series is a one dimensional datastructure object, dataframe is a multi-dimensional data structure object.
4. A Series object is value mutable.
5. A Series object is size mutable.
6. A DataFrame object is value mutable.
7. A DataFrame object is size mutable.
8. There is no difference between a NumPy array and a Series object.
9. A DataFrame can be thought of as a group of multiple Series objects.

Q1.

Given a Series s13

A 7000

B 6000

C 6000

D 2000

What **is** the error?

```
import pandas as pd
s13.index=range(0,5)
print(s13)
```

Q2. Correct the error

Q3. Write the output of the following 2 codes-

```
import pandas as pd
import numpy as np
fst=[9,10,11]
ob1=pd.Series(data=fst*2)
print(ob1)
```

```
fst=pd.Series(data=[9,10,11])
ob2=pd.Series(data=fst*2)
print(ob2)
```

Q4. Write the output-

DATA SERIES

```
data=np.array(['a1','b1','c1','d1','e1','f1'])
s=pd.Series(data)
print("I.")
print(s[:3])
print("II.")
print(s[-3:])
```

ANSWERS-

A1. The number of index and elements in the ds obj don't match

A2. range(0,4) or range(1,5)

A3.

```
import pandas as pd
import numpy as np
fst=[9,10,11]
ob1=pd.Series(data=fst*2)
print(ob1)
```

```
0    9
1   10
2   11
3    9
4   10
5   11
dtype: int64
```

```
fst=pd.Series(data=[9,10,11])
ob2=pd.Series(data=fst*2)
print(ob2)
```

```
0    18
1    20
2    22
dtype: int64
```

A4.

```
data=np.array(['a1','b1','c1','d1','e1','f1'])
s=pd.Series(data)
print("I.")
print(s[:3])
print("II.")
print(s[-3:])
```

```
I.
0    a1
1    b1
2    c1
dtype: object
II.
3    d1
4    e1
5    f1
dtype: object
```

II) DATA FRAME DATA STRUCTURE

Def

DataFrame- It is a **2D labeled array like** pandas data structure that stores an ordered collection columns that can store data of diff. types.

A 2D array is an array having single-dimension arrays as its elements.

Eg-If no. of elements in an array $a[7][9]$ is 7rowsx9 cols=63

ds	series
0	1
1	2
2	3

dtype: int64

df	dataframe
0	1
1	2
2	3

Notice the col index

Data values

MAJOR CHARACTERISTICS OF A DATAFRAME- pg(31)

- 1) It has 2 axes-a row index(axis0) and a column index(axis1)
- 2) Each value is identified with a row index and a column index. The row index is called **index** and the column index is called **columns**.
- 3) The indexes can be letters, numbers or strings.
- 4) Columns can have data of a different types.
- 5) It is value mutable
- 6) It is size mutable

	NAME	AGE	RURAL	URBAN
0	Abc	80	876	1123
1	Xyz	45	NaN	765
2	Pqr	NaN	543	NaN

Row index axis=0

column index Axis=1

Missing values

SUMMARY (DF)part

III) CREATING A DATA FRAME

Syntax- `df=pd.DataFrame(<2D datastructure>,[index],[columns])`

1)Creating a df object from a 2-D Dictionary

- 
- a) Creating a dataframe from a 2D dictionary having values as lists/ndarrays/series
 - b) Creating a dataframe from a 2D dictionary having values as dictionaries.

2) Creating a DataFrame Object from a List of Dictionaries/Lists(2D List)

3) Creating a DataFrame Object from a 2-D array

4) Creating a dataframe from a 2D dictionary having values as series.

5) Creating a DataFrame object from another DataFrame object-

IV) DATAFRAME ATTRIBUTES

V) SELECTING OR ACCESSING DATA-

- i) Selecting or accessing a column
- ii) Selecting/Accessing multiple columns
- iii)Selecting/Accessing a subset from a DF using Row/Col

names

- a) To access a row
- b) To access multiple rows
- c) To access subset cols
- d) To access a range of cols from a range of rows
- iv) Selecting rows/cols from a DF

v) Selecting or accessing Individual values

VI) ADDING/MODIFYING ROWS/COLS VALUES IN DF

i) Adding/modifying a column

ii) Adding/modifying a row

iii)Modifying a single cell

VII) DELETING/RENAMING COLUMNS/ROWS in a DF

i)Deleting rows/cols

a) Deleting a column using del

b) Deleting a row using drop

c) Deleting row/s col/s using drop

ii) Renaming rows/cols

VIII) MORE ON DF INDEXING-BOOLEAN INDEXING

i) Creating df with Boolean indexes

ii) Accessing rows from DF with Boolean Indexes

same

III) CREATING AND DISPLAYING A DATAFRAME

- A dataframe object can be created by passing data in a 2D format.
- We need to import both pandas and numpy

Syntax- `<dataframe obj>=panda.DataFrame(<2D data structure>,[columns=<col sequence>],[index=<index sequence>])`

Df=pd.DataFrame() creates an empty df having no rows and no columns

2D data structures could be made up of

- 2D dictionary i.e. dictionaries having lists or dictionaries or ndarrays or Series objects etc.
- 2D ndarrays (NumPy Arrays)
- Series type Object
- Another df object.

1)Creating a df object from a 2-D Dictionary-

What is a 2-d dictionary?

```
>>> d={'name':'gita'}
>>> d
{'name': 'gita'}
```

1D
dictionary

```
>>> d={'name':'gita'}
>>> df=pd.DataFrame(d);
```

ValueError: If using all scalar values, you must pass an index

```
>>> d={'name':['gita','sita','sita']}
>>> d
{'name': ['gita', 'sita', 'sita']}
```

2D
dictionary

```
>>> df=pd.DataFrame(d);
>>> df
   name
0  gita
1  sita
2  sita
```

No error

Now, no error
since we
converted it to a
2-D dictionary

```
>>> df=pd.DataFrame(d,index=['a'])
>>> df
   name
a  abc
```

gita

so

```
>>> d={'name':['gita']}
>>> d
{'name': ['gita']}
>>> df=pd.DataFrame(d);
>>> df
   name
0  gita
```

1)Creating a df object from a 2-D Dictionary-

A 2D dictionary is a dictionary having items as (key,value) where value part is a data structure of any type

-another dictionary

-an ndarray

-a Series Object

-a list

but the value part of the keys should have similar structure and **equal length**.

a) Creating a dataframe from a 2D dictionary having values as lists/ndarrays-

Prog-1 Create a df----->

```
df=pd.DataFrame({'Names':['abc','xyz'],'Age':[10,20],'Marks':[10]})
ValueError: arrays must all be same length
>>> df=pd.DataFrame({'Names':['abc','xyz'],'Age':[10,20],'Marks':[10,20]})
>>> df
   Names  Age  Marks
0   abc   10    10
1  xyz   20    20
```

Note-if the 2d dictionary has values as lists and if the length of the lists don't match. You will get an error.

Not of equal length

	Marks	Sport	Student
0	70	Cricket	Rahul
1	80	Badminton	Neha
2	90	Football	Mark
3	100	Athletics	Smith

Now of Equal length


```
#Prog1
import pandas as pd
import numpy as np
d={'Sport':['Cricket','Badminton','Football','Athletics'],'Student':['Rahul','Sneha','Mark','Smith'],'Marks':[70,80,90,100]}
print(d)
print("-----")
df=pd.DataFrame(d)
print(df)
```

	Marks	Sport	Student
0	70	Cricket	Rahul
1	80	Badminton	Neha
2	90	Football	Mark
3	100	Athletics	Smith

```
{'Sport': ['Cricket', 'Badminton', 'Football', 'Athletics'], 'Student': ['Rahul', 'Sneha', 'Mark', 'Smith'], 'Marks': [70, 80, 90, 100]}
```

	Sport	Student	Marks
0	Cricket	Rahul	70
1	Badminton	Sneha	80
2	Football	Mark	90
3	Athletics	Smith	100

```
#eg
df=pd.DataFrame({'Qtr1':[100,200,300],'Qtr2':[500,200,300],'Qtr3':[6,4,6]})
df
```

	Qtr1	Qtr2	Qtr3
0	100	500	6
1	200	200	4
2	300	300	6

Dictionary values as lists

Not for me

*Its index is assigned automatically 0 onwards and columns created from keys are placed **in sorted order**.

*keys of dictionary have become columns of df

* You can specify your own sequence for the index

Prog2-same df as prog1 but with indexes as I,II,III,IV

```
#prog2
df1=pd.DataFrame(d,index=['I','II','III','IV'])
print(df1)
```

	Sport	Student	Marks
I	Cricket	Rahul	70
II	Badminton	Sneha	80
III	Football	Mark	90
IV	Athletics	Smith	100

The number of indexes should match with the length of the dictionary values else python will give an error

```
#Eg
df1=pd.DataFrame(d,index=['I','II','III','IV','V'])
print(df1)
```

ValueError: Shape of passed values is (4, 3), indices imply (5, 3)

Prog-3 same df as Prog1 above but this time the dictionary values will be ndarrays-

Note- If length of the inner nparray not same then error!!!

```
import numpy as np
import pandas as pd
ar1=np.array([1,2,3])
ar2=np.array([11,22])
ar3=np.array([4,6,31])
dff=pd.DataFrame({'Score':ar1,'Age':ar2,'Marks':ar3})
dff
```

ValueError: arrays must all be same length

```
#prog3
ar1=np.array([70,80,90,100])
ar2=np.array(['Cricket','Badminton','Football','athletics'])
ar3=np.array(['Rahul','Sneha','Mark','Smith'])
d={'Marks':ar1,'Sport':ar2,'Student':ar3}
df=pd.DataFrame(d)
print(df)
```

	Marks	Sport	Student
0	70	Cricket	Rahul
1	80	Badminton	Sneha
2	90	Football	Mark
3	100	athletics	Smith

Prog3_a- Create the following df using dictionaries and its values as Series.

	Age	Marks	Name
0	1	4	Aba
1	2	5	Xyz
2	3	6	pqr

```
import pandas as pd
import numpy as np
s1=pd.Series([1,2,3])
s2=pd.Series([4,5,6])
s3=pd.Series(["Aba","Xyz","pqr"])
df=pd.DataFrame({"Age":s1,"Marks":s2,"Name":s3})
df
```

Note-

If we use a 2d dictionary to create a df
With its values as series and if the
length of the series is not equal, no
error, NaN's are added wherever
required.

```
>>> s1=pd.Series([1,2,3])
>>> s2=pd.Series([1,2])
>>> df=pd.DataFrame({"Age":s1,"No":s2})
>>> df
```

	Age	No
0	1	1.0
1	2	2.0
2	3	NaN

If length is not the same

List	}	Errors
Ndarray		
Dictionary	}	No errors
Series		

Prog-4 pg(34) Eg21

EXAMPLE 21 Given a dictionary that stores the section names' list as value for 'Section' key and contribution amounts' list as value for 'Contri' key :

```
dict1 = {'Section': [ 'A', 'B', 'C', 'D'],
        'Contri': [ 6700, 5600, 5000, 5200] }
```

Write code to create and display the data frame using above dictionary.

```
SOLUTION import pandas as pd
dict1 = {'Section': [ 'A', 'B', 'C', 'D'],
        'Contri': [ 6700, 5600, 5000, 5200] }
df1 = pd.DataFrame(dict1)
print(df1)
```

Output

	Contri	Section
0	6700	A
1	5600	B
2	5000	C
3	5200	D

2D dictionary with values as lists.

b) Creating a dataframe from a 2D dictionary having values as dictionaries-

Prog-5 **Note** -if u are using a 2d dictionary with values as dictionaries to make a df and if the length of the inner dictionary don't match then no error, NaN will be put in that place.

Outer dictionary keys as columns

```
: #Prog5
import pandas as pd
d1={'Marketing':{'Age':25,'Name':'Neha','Gender':'Female'},'Sales':{'Age':24,'Name':'Rohit','Gender':'Male'}}
df=pd.DataFrame(d1)
print(df)
```

	Marketing	Sales
Age	25	24
Name	Neha	Rohit
Gender	Female	Male

	2015	2016
Qtr1	100	300
Qtr2	200	100
Qtr3	300	200

```
#Eg
d1={'Qtr1':100,'Qtr2':200,'Qtr3':300}
d2={'Qtr1':300,'Qtr2':100,'Qtr3':200}
df=pd.DataFrame({2015:d1,2016:d2})
df
```

	2015	2016
Qtr1	100	300
Qtr2	200	100
Qtr3	300	200

Inner dictionary keys as indexes

[Try it yourself](#)- create the foll data frame using a 2D dictionaries with values as

- 1)Lists
- 2)Ndarrays
- 3)Dictionaries

	Col1	Col2
One	1	11
Two	2	11
Three	3	22

```
df=pd.DataFrame({"Col1":np.array([1,2,3]),"Col2":np.array([11,11,22])},index=["One","Two","Three"])
df
```

	Col1	Col2
One	1	11
Two	2	11
Three	3	22

2.

1.

```
df=pd.DataFrame({"Col1":[1,2,3],"Col2":[11,11,22]},index=["One","Two","Three"])
df
```

	Col1	Col2
One	1	11
Two	2	11
Three	3	22

```
import pandas as pd
import numpy as np
df=pd.DataFrame({"Col1":{"One":1,"Two":2,"Three":3},"Col2":{"One":11,"Two":11,"Three":22}})
df
#or
d1={"One":1,"Two":2,"Three":3}
d2={"One":11,"Two":11,"Three":22}
df=pd.DataFrame({"Col1":d1,"Col2":d2})
df
```

3.

Prog-6 Create a df from a 2D dictionary, Sales, which stores the quarter-wise sales as inner dictionary for 2 years

Pg(34)Eg22

```
#Prog6
d1={'yr1':{'Qtr1':34500,'Qtr2':56000,'Qtr3':47000,'Qtr4':49000},'yr2':{'Qtr1':44900,'Qtr2':46100,'Qtr3':57000,'Qtr4':59000}}
df=pd.DataFrame(d1)
df
```

Prog7 pg(35 Eg23)

	yr1	yr2
Qtr1	34500	44900
Qtr2	56000	46100
Qtr3	47000	57000
Qtr4	49000	59000

Eg-if length of the inner dictionary values don't match

```
df=pd.DataFrame({'A':{'roll':1,'name':'aaa'},'B':{'roll':2}})
df
```

	A	B
roll	1	2.0
name	aaa	NaN

EXAMPLE 23 Carefully read the following code.

1

Qtr1	44900	NaN
Qtr2	46100	NaN
Q3	57000	NaN
Q4	59000	NaN
A	NaN	54500
B	NaN	51000
Qtr4	NaN	57000

2

```
import pandas as pd
yr1 = { 'Qtr1' : 44900, 'Qtr2' : 46100, 'Q3' : 57000, 'Q4' : 59000 }
yr2 = { 'A' : 54500, 'B' : 51000, 'Qtr4' : 57000 }
diSales1 = { 1 : yr1, 2 : yr2 }
df3 = pd.DataFrame(diSales1)
```

(i) list the index labels of the DataFrame df3

(ii) list the column names of DataFrame df3.

SOLUTION

(i) The index labels of df3 will include : A, B, Q3, Q4, Qtr1, Qtr2, Qtr4.

The total number of indexes is equal to total unique inner keys, i.e., 7.

(ii) The column names of df3 will be : 1, 2

df3.index

df3.columns

- Summary-**
- Creating df with a 2d dictionary with values
- 1.List

2.Ndarray

3.Series

4.dictionary
- If length not same error

If length not same error

If length not same no error

If length not same no error

2)Creating a DataFrame Object from a List of **Dictionaries/Lists**(2D List)→no errors with diff lengths of inner seq

---List of Lists

Prog-8

```
#prog8
df=pd.DataFrame([[1,2,3],[4,5,6]])
df
```

Default index

	0	1	2
0	1	2	3
1	4	5	6

```
df=pd.DataFrame([[1,2,3],[4,5,6]],index=['A','B'],columns=['Roll','Age','Marks'])
df
```

	Roll	Age	Marks
A	1	2	3
B	4	5	6

No error→
Even if the length of the inner list is not the same

```
>>> df=pd.DataFrame([[1,2,3],[5,6]])
>>> df
   0  1  2
0  1  2  3.0
1  5  6  NaN
```

---List of Dictionaries

The dictionary keys will become columns and the inner dictionary values will become rows.

Eg(don't copy)→

```
>>> l=[{'name':'abc','age':20},{ 'name':'abb','age':20},{ 'name':'ddd','age':20}]
>>> df=pd.DataFrame(l)
>>> df
   name  age
0  abc   20
1  abb   20
2  ddd   20
```


If the dictionaries in the list have diff lengths, no error

Eg→
Don't copy

```
import pandas as pd
import numpy as nd
d1={"Marks":1,"Age":2}
d2={'Marks':5,"Age":3}
d3={'Marks':3}
d4={"Marks":8,"pin":9}
l=[d1,d2,d3,d4]
df=pd.DataFrame(l)
df
```

	Marks	Age	pin
0	1	2.0	NaN
1	5	3.0	NaN
2	3	NaN	NaN
3	8	NaN	9.0

If the ndarrays in the list have diff lengths, no error

```
import numpy as np
import pandas as pd
dff=pd.DataFrame([np.array([1,2,3]),np.array([7,8])])
dff
```

	0	1	2
0	1	2	3.0
1	7	8	NaN

Prog-9

```
#prog9
topper1={'Rollno':1,'Name':'Mita','Marks':99}
topper2={'Rollno':7,'Name':'Gita','Marks':98}
topper3={'Rollno':4,'Name':'Sita','Marks':97}
list=[topper1,topper2,topper3]
df=pd.DataFrame(list)
df
```

	Rollno	Name	Marks
0	1	Mita	99
1	7	Gita	98
2	4	Sita	97

```
df=pd.DataFrame(list,index=['SecA','SecB','SecC'])
df
```

	Rollno	Name	Marks
SecA	1	Mita	99
SecB	7	Gita	98
SecC	4	Sita	97

Prog10-pg(36)Eg24

Prog11-pg(37)Eg25

Prog12-pg(37)Eg26

```
#Prog10
d1={'Target':10,'Sales':20}
d2={'Target':20,'Sales':30}
d3={'Target':60,'Sales':70}
d4={'Target':20,'Sales':88}
list=[d1,d2,d3,d4]
df=pd.DataFrame(list,index=['zoneA','zoneB','zoneC','zoneD'])
df
```

	Target	Sales
zoneA	10	20
zoneB	20	30
zoneC	60	70
zoneD	60	70

```
>>> d1={'Name':'aaa','Marks':20,'Age':10}
>>> d2={'Name':'aaa','Marks':20}
>>> df=pd.DataFrame([d1,d2])
>>> df
   Name  Marks  Age
0  aaa     20  10.0
1  aaa     20   NaN
```

If length of dictionaries don't match, no error, a NaN will be put in that place.

Prog-11

EXAMPLE 25 Write a program to create a dataframe from a 2D list. Specify own index labels.

SOLUTION

```
import pandas as pd
list2 = [ [ 25, 45, 60], [34, 67, 89], [88, 90, 56] ]
df2 = pd.DataFrame(list2, index = ['row1', 'row2', 'row3'] )
print ( df2 )
```

Output

	0	1	2
row1	25	45	60
row2	34	67	89
row3	88	90	56

Prog-12 wap to create a dataframe from a list containing 2 lists, each containing Target and Sales figures of 4 zonal offices. Give appropriate row labels.

Zone A Zone B Zone C Zone D

Target	1	3	5	7
Sales	2	4	6	8

```
df = pd.DataFrame([ [1, 3, 5, 7], [2, 4, 6, 8] ], index = ['Target', 'Sales'], columns = ['Zone A', 'Zone B', 'Zone C', 'Zone D'])
```

3) Creating a DataFrame Object from a 2D ndarray

You can also pass a **2D Numpy** array having shape(<n>,<n>) to a DataFrame() to create a dataframe Object.

Prog14-
Default
index
and
columns

```
#Prog14
arr1=np.array([[1,2,3],[4,5,6]])
print(arr1.shape)
df=pd.DataFrame(arr1)
df
```

(2, 3)

	0	1	2
0	1	2	3
1	4	5	6

Prog15

```
#Prog14
arr1=np.array([[1,2,3],[4,5,6]])
print(arr1.shape)
df=pd.DataFrame(arr1,columns=['One','Two','Three'])
df
```

(2, 3)

	One	Two	Three
0	1	2	3
1	4	5	6

Give columns

Prog16-

```
arr=np.array([[11.6,21.2,33.5],[40,50,60],[212,44.5,23.4]])
df=pd.DataFrame(arr,columns=['First','Second','Third'],index=['A','B','C'])
df
```

	First	Second	Third
A	11.6	21.2	33.5
B	40.0	50.0	60.0
C	212.0	44.5	23.4

-Ndarrays that are passed to DataFrame have same number of elements in each of the rows.

-If rows of ndarrays differ in length(if the number of elements in each row differ, then Python will create a **single column** in the dataframe and the datatype in the column will be **object**.

```
arr=np.array([[1,2,3],[1,2,3,4],[1,2,3,]])  
df=pd.DataFrame(arr)  
df
```

```
arr=np.array([[1,2,3],[1,2,3,4],[1,2,3,]])  
df=pd.DataFrame(arr,index=['A','B','C'],columns=['One'])  
df
```

	0
0	[1, 2, 3]
1	[1, 2, 3, 4]
2	[1, 2, 3]

Same as
[1,2,3]

	One
A	[1, 2, 3]
B	[1, 2, 3, 4]
C	[1, 2, 3]

column and index from the user.

Prog-18 pg(39) Eg27

Prog19 pg(39) Eg28

EXAMPLE 27 What will be the output of following code ?

```
import pandas as pd
import numpy as np
arr1 = np.array([ [11, 12], [13, 14 ], [15, 16] ], np.int32)
dtf2 = pd.DataFrame(arr1)
print(dtf2)
```

SOLUTION

```
   0  1
0  11 12
1  13 14
2  15 16
```

```
   0  1
0  11 12
1  13 14
2  15 16
```

EXAMPLE 28 Write a program to create a DataFrame from a 2D array as shown below :

101	113	124
130	140	200
115	216	217

SOLUTION

```
import pandas as pd
import numpy as np
arr2 = np.array([ [101, 113, 124], [130, 140, 200 ], [115, 216, 217] ])
dtf3 = pd.DataFrame(arr2)
print(dtf3)
```

Output

```
   0  1  2
0  101 113 124
1  130 140 200
2  115 216 217
```

4) Creating a DataFrame Object from a 2D Dictionary with values as Series Objects

You can create a DF obj by using multiple series objects. In a 2D dictionary, u can have the value parts as series objects and then pass this dictionary as argument to create a DF object.

```
#Eg
import numpy as np
import pandas as pd
s1=pd.Series([1,2,3])
s2=pd.Series([4,5,6])
df=pd.DataFrame({'One':s1, 'Two':s2})
df
```

	One	Two
0	1	4
1	2	5
2	3	6

Prog-20

```
#Prog20
staff=pd.Series([20,36,44])
salaries=pd.Series([16000,24000,56000])
d={'people':staff, 'Amount':salaries}
df=pd.DataFrame(d)
df
```

	people	Amount
0	20	16000
1	36	24000
2	44	56000

EXAMPLE 29 Consider two series objects *staff* and *salaries* that store the number of people in various office branches and salaries distributed in these branches, respectively.

Write a program to create another Series object that stores average salary per branch and then create a DataFrame object from these Series objects.

★ Series allows vectorized operations.

SOLUTION `import pandas as pd`

Arrays also allow vectorized operations

`import numpy as np`

`staff= pd.Series([20, 36, 44])`

`salaries = pd.Series([279000, 396800, 563000])`

`avg = salaries / staff` # it will create avg series object

`org= {'people':staff, 'Amount':salaries, 'Average':avg }`

`dtf5 = pd.DataFrame(org)`

`print(dtf5)`

Output

	Amount	Average	people
0	279000	13950.000000	20
1	396800	11022.222222	36
2	563000	12795.454545	44

5) Creating a DataFrame object from another DataFrame object-

`df1=pd.DataFrame(df)` or `df1=df` -any change in one reflects the other.

Or `df2=df.copy()` -any change in one does not reflect the other.

Prog22-

```
df1=pd.DataFrame(df)
df1
```

	people	Amount
0	20	16000
1	36	24000
2	44	56000

```
df1['Sal']=100
df1
```

	people	Amount	Sal
0	20	16000	100
1	36	24000	100
2	44	56000	100

```
df
```

	people	Amount	Sal
0	20	16000	100
1	36	24000	100
2	44	56000	100

*note-DF's can also be created from text/csv files.

```
df2=df.copy()
df2
```

	people	Amount	Sal
0	20	16000	100
1	36	24000	100
2	44	56000	100

```
df2['comm']=20
df2
```

	people	Amount	Sal	comm
0	20	16000	100	20
1	36	24000	100	20
2	44	56000	100	20

```
df
```

	people	Amount	Sal
0	20	16000	100
1	36	24000	100
2	44	56000	100

IV) DATAFRAME OBJECT ATTRIBUTES

slide31(attributes of ds)

When you create a DF obj, all information related to it is available through its attributes. You can use these attributes in the following format

Syntax **<DF obj>.<attribute name>** **TB pg(41)**

Attribute	Description
index	The index row labels of the DataFrame(sequence)
columns	The column labels of the Dataframe(sequence)
axes	Returns a list representing both the axes(axis 0 and axis 1) of the Dataframe
dtypes	Returns the dtypes of data in the DF (column wise)
size	Returns an int of the number of elements in the df obj
shape	Returns a tuple representing the dimensions of the df
values	Returns a numpy representation of the dataframe
empty	Indicator whether DataFrame is empty
ndim	Returns an int representing the number of axes/array dimensions
T	Transpose index and columns.

We will be using this df for all attribute programs-

```
#Prog23
import numpy as np
import pandas as pd
dfn=pd.DataFrame([[25,24],['Neha','Rohit'],['Female','Male']],index=['age','name','gender'],columns=['Marketing','Sales'])
dfn
```

	Marketing	Sales
age	25	24
name	Neha	Rohit
gender	Female	Male

(a)Retrieving various properties of a Df Object- dfn.index

Prog-23-

```
dfn.index
Index(['age', 'name', 'gender'], dtype='object')
```

```
dfn.columns
Index(['Marketing', 'Sales'], dtype='object')
```

dfn.columns dfn.axes dfn.dtypes

```
dfn.axes
[Index(['age', 'name', 'gender'], dtype='object'),
Index(['Marketing', 'Sales'], dtype='object')]
```

```
dfn.dtypes
Marketing    object
Sales        object
dtype: object
```

datatype is listed for individual columns.

Another eg of dtypes

	0	1	2	Sal	comm
A	1	2	3	100	10
B	4	5	6	200	20

```
>>> df.dtypes
0      int64
1      int64
2      int64
Sal    int64
comm   int64
dtype: object
```



? →

```
>>> s2=df.dtypes
>>> s2
0      int64
1      int64
2      int64
Sal    int64
comm   int64
dtype: object
>>> s2[0]
dtype('int64')
```

Object dt

(b)Getting number of rows in a DF→len(df)

len(<Df object>) will return the number of rows in a dataframe or len(dfn.index) or dfn.shape[0]

Eg:- len(dfn)---→3

	Marketing	Sales
age	25	24
name	Neha	Rohit
gender	Female	Male

(c)Getting count of non-Na values in DF

Like series ,u can use count() with a DF to get the count of non-NaN values,but count with a DF is a little elaborate-

dfn.count()-for each column
dfn.count(1)-for each row

i) If u don't pass any argument or pass 0(default 0),then it returns count of non-NaN values for each column.

Prog-24

```
: dfn.count()
#dfn.count(0)
#dfn.count(axis='index')
#dfn.count(axis=0)

: Marketing    3
  Sales       3
  dtype: int64
```

ii) If u pass argument as 1,then it returns count of non-Na values for each row

Prog-25

eg >>> df=pd.DataFrame([[1,2,3],[4,5,None]])
>>> df
 0 1 2
0 1 2 3.0
1 4 5 NaN
>>> len(df)
2
>>> df.size
6
>>> df.count()
0 2
1 2
2 1
dtype: int64

```
: dfn.count(1)
#dfn.count(axis=1)
#dfn.count(axis='columns')
```

```
: age      2
  name     2
  gender   2
  dtype: int64
```

(d) Transposing a DF-→Df.T

You can transpose a DF by swapping its indexes and columns by using attribute T

Prog-26

```
#prog26
dfn.T
```

	age	name	gender
Marketing	25	Neha	Female
Sales	24	Rohit	Male

	Marketing	Sales
age	25	24
name	Neha	Rohit
gender	Female	Male

Prog-27 pg(43)Eg30

	Weight	Age	Name
0	40	15	Rohit
1	50	17	Sahil
2	37	14	Rina

EXAMPLE 30 Write a program to create a DataFrame to store weight, age and names of 3 people. Print the DataFrame and its transpose.

SOLUTION

```
import pandas as pd
# Creating the DataFrame
df = pd.DataFrame({'Weight': [42, 75, 66],
                  'Name': ['Arnav', 'Charles', 'Guru'],
                  'Age': [15, 22, 35]})
print('Original DataFrame')
print(df)
print('Transpose:')
print(df.T)
```

Handwritten code:
df = pd.DataFrame([[40, 15, 'Rohit'], [50, 17, 'Sahil'], [37, 14, 'Rina']], columns=['Weight', 'Age', 'Name'])
print(df)
print(df.T)

Output

```
Original DataFrame
   Age  Name  weight
0   15  Arnav    42
1   22 Charles    75
2   35   Guru    66

Transpose:
      0      1      2
Age   15   22   35
Name  Arnav Charles Guru
weight 42    75    66
```

NOTE

You can also use **shape[0]** to see the number of rows and **shape[1]** for getting number of columns, i.e.,

```
df.shape[0]
df.shape[1]
```

(e) Retrieving size, shape, no. of dimensions of the DF object-

dfn.size-returns the no. of elements in the df obj

dfn.shape-returns a tuple giving the no. of rows and columns in a tuple form

dfn.ndim-returns the no. of dimensions of the DF object as an int

	Marketing	Sales
age	25	24
name	Neha	Rohit
gender	Female	Male

Prog-28

```
#Prog28
print(dfn.size)
print(dfn.shape)
print(dfn.ndim)
print(dfn.shape[0])# to get the number of rows
print(dfn.shape[1])# to get the number of columns
```

```
6
(3, 2)
2
3
2
```

(f) Numpy Representaion of DataFrame-

You can represent the values of a Df object in Numpy way using-

Prog-29

```
#Prog29
dfn.values

array([[25, 24],
       ['Neha', 'Rohit'],
       ['Female', 'Male']], dtype=object)
```


(g) Checking for empty df-

A df is said to be empty if its any axes(0 or 1) has no values

Having np.NaN does not mean empty

```
#Eg
df=pd.DataFrame()
df.empty
```

True

```
#Eg
df=pd.DataFrame(columns=['A'])
print(df)
print(df.size)
df.empty
```

Empty DataFrame
Columns: [A]
Index: []
0

True

```
#Eg
df=pd.DataFrame(index=['a','b'])
print(df)
print(df.size)
print(df.empty)
```

Empty DataFrame
Columns: []
Index: [a, b]
0

True

1.11.1 Dataframe vs. Series

Dataframe as such is a 2D data structure while Series is a 1D data structure and Series is value-mutable only while a dataframe is value-mutable as well as size-mutable.

pg44

Table 1.7 Dataframe vs. Series

S.No.	Dataframe Object	Series Object
1.	It is a 2D data structure.	It is a 1D data structure.
2.	It is value-mutable as well as size-mutable.	It is value-mutable only.
3.	It can store heterogeneous elements (different datatypes).	It stores homogeneous elements (same datatype).
4.	Each column has a heading.	Its single column does not have a heading.

Each individual column of a dataframe can be considered equivalent to that of a series object. A dataframe with a single column is similar to a series object with a little difference that now it also has size-mutability unlike Series objects.²

1.11.2 Dataframe vs. 2D Narray

A Dataframe, like a 2D ndarray is a two-dimensional data structure. However, it is different from 2D ndarrays.

Let us see how.

Table 1.8 Dataframe vs. 2D Narrays

S.No.	Dataframe Object	2D Narrays
1.	It is a 2D data structure.	It is also a 2D data structure.
2.	It can store heterogeneous elements (different datatypes).	It is a table of homogenous elements (usually numbers), all of the same type.
3.	It can have indexes as well as labels for rows and columns.	It is indexed by a tuple of positive integers for both axes.
4.	It consumes more memory than an equivalent ndarray.	It consumes lesser memory compared to equivalent dataframe.
5.	Dataframes are expandable as you can add new elements in the same dataframe.	NumPy arrays are not expandable. If you add new elements then a new array will be created. Even with <code>append()</code> function a new array is created and filled.

V) SELECTING OR ACCESSING DATA-

From a df you can extract or select desired rows and columns-

dtf5→

```
: #Eg
import numpy as np
import pandas as pd
dtf5=pd.DataFrame({'Population':[56789,80000,75000,46000], 'Hospitals':[190,300,150,350], 'Schools':[7000,10000,8000,9000]},
                  index=['Delhi', 'Mumbai', 'Kolkata', 'Chennai'])
dtf5
```

:

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

i) Selecting/Accessing a column-

<df>.colname -> no single quotes here

<df>['colname']

Prog-30-write the output of the following-

```
#Prog30
print(dtf5['Population'])
#print(dtf5.Population)
print()
print(dtf5['Schools'])
#print(dtf5.Schools)
```

```
Delhi      56789
Mumbai     80000
Kolkata    75000
Chennai    46000
Name: Population, dtype: int64
```

```
Delhi      7000
Mumbai     10000
Kolkata     8000
Chennai     9000
Name: Schools, dtype: int64
```

ii) Selecting/Accessing Multiple Columns(selective cols)

You can give a list of columns inside square brackets with df objects-

Syntax- <df obj>[[colname,colname,colname,.....]]

Prog-31 Write the output-



	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

```
#Prog31
dtf5[['Population','Hospitals']]
```

	Population	Hospitals
Delhi	56789	190
Mumbai	80000	300
Kolkata	75000	150
Chennai	46000	350

```
dtf5[['Hospitals','Population']]
```

	Hospitals	Population
Delhi	190	56789
Mumbai	300	80000
Kolkata	150	75000
Chennai	350	46000

EXAMPLE 31 Given a DataFrame namely *aid* that stores the aid by NGOs for different states :

	Toys	Books	Uniform	Shoes
Andhra	7916	6189	610	8810
Odisha	8508	8208	508	6798
M.P.	7226	6149	611	9611
U.P.	7617	6157	457	6457

Write a program to display the aid for

- (i) Books and Uniform only (ii) Shoes only

SOLUTION

```
import pandas as pd
:      # DataFrame aid created or loaded
print("Aid for books and uniform:")
print(aid[['Books', 'Uniform']])
print("Aid for shoes:")
print(aid.Shoes) or aid['shoes']
```

Output

```
Aid for books and uniform:
      Books  Uniform
Andhra  6189      610
Odisha  8208      508
M.P.    6149      611
U.P.    6157      457
Aid for shoes:
Andhra   8810
Odisha   6798
M.P.     9611
U.P.     6457
Name: Shoes, dtype: int64
```

iii) Selecting/Accessing a Subset from a DF using row/col names

.loc always begin from a row

.loc end row and end col are inclusive

Syntax-<DF>.loc[startrow:endrow,startcolumn:end column]

row

column

Both end indexes are inclusive for loc

loc always works with labels

(a) To access a row- just give the row label/name as

<df>.loc[<row label>,:] ->best(prefer don't miss the comma and colon)

<df>.loc[<row label>,]

<df>.loc[<row label>]

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

Prog33-Access the Delhi row in diff ways

```
: #prog-33
dtf5.loc['Delhi']
#dtf5.loc['Delhi',:]
#dtf5.loc['Delhi']
```

dtf5.loc['Delhi' : , :]-then all rows will follow in the output(entire df)

```
: Population      56789
Hospitals         190
Schools           7000
Name: Delhi, dtype: int64
```

(b) To access multiple rows-

<df>.loc[<start row>:<end row>,:]

Prog34 Display the rows of Mumbai and Kolkata

dtf5.loc[[rowname,rowname,rowname]]

dtf5.loc[['Mumbai','Kolkata']]

```
dtf5.loc['Mumbai':'Kolkata']
#dtf5.loc['Mumbai':'Kolkata',:]
#dtf5.loc['Mumbai':'Kolkata',]
```

	Population	Hospitals	Schools
Mumbai	80000	300	10000
Kolkata	75000	150	8000

```
df=pd.DataFrame([[1,2,3],[6,7,8],[9,8,7]],index=['a','b','c'])
df
```

	0	1	2
a	1	2	3
b	6	7	8
c	9	8	7

```
df.loc['a':,:]
```

	0	1	2
a	1	2	3
b	6	7	8
c	9	8	7

```
df.loc[['a','c']]
```

	0	1	2
a	1	2	3
c	9	8	7

Prog35 write the output

```
: dtf5.loc['Mumbai':'Chennai']  
#dtf5.loc['Mumbai':'Chennai',:]  
#dtf5.loc['Mumbai':'Chennai',]
```

:

	Population	Hospitals	Schools
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

(c) To access subset of columns-

Syntax- <df obj>.loc[:,<start column>:<end column>] -> don't miss the colon and comma

Prog-36 Write the output-

```
dtf5.loc[:, 'Population':'Schools']
```

```
dtf5.loc[:, 'Population':'Hospitals']
```

Multiple columns

df[[col1,col2,col3,.....]]

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

	Population	Hospitals
Delhi	56789	190
Mumbai	80000	300
Kolkata	75000	150
Chennai	46000	350

(d) To access range of columns from a range of rows-

Prog-38 write the output

```
dtf5.loc['Delhi':'Mumbai','Population':'Hospitals']
```

	Population	Hospitals
Delhi	56789	190
Mumbai	80000	300

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

Prog-39 pg(48) Eg32

EXAMPLE 32 Given a DataFrame namely *aid* that stores the aid by NGOs for different states :

	Toys	Books	Uniform	Shoes
Andhra	7916	6189	610	8810
Odisha	8508	8208	508	6798
M.P.	7226	6149	611	9611
U.P.	7617	6157	457	6457

Write a program to display the aid for states 'Andhra' and 'Odisha' for Books and Uniform only.

SOLUTION

```
import pandas as pd
# DataFrame aid created or loaded
print( aid.loc['Andhra':'Odisha', 'Books':'Uniform'])
```

Output

	Books	Uniform
Andhra	6189	610
Odisha	8208	508

iv) Selecting rows/columns from a DataFrame-

Sometimes your df may not contain row and column labels or you may not remember them.in such cases you can extract subset from dataframe using the row and column numeric position , but this time you will use iloc instead of loc.

iloc means integer location

Syntax- **<df>.iloc[start row index : end row index , start column index : end col index]**

just like slicing



Prog-40 Write the output-

```
#Prog40
dtf5.iloc[0:2,1:3]
```

**df.iloc[]-works on position only only only
like
ds slicing also works on position only only**

Both end indexes are exclusive

	Hospitals	Schools
Delhi	190	7000
Mumbai	300	10000

```
dtf5.iloc[0:2,1:2]
```

iloc[0:2,1:1] ?

Delhi
Mumbai

	Hospitals
Delhi	190
Mumbai	300

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

EXAMPLE 33 Consider a dataframe `df` as shown below.

Sample DataFrame `df` (Reference 1.7a)

	Item Type	Sales Channel	Order Date	Order ID	Total Revenue	Total Cost	Total Profit
0	Cosmetics	Online	5/22/2017	8985	793518.00	477943.95	315574.05
1	Cereal	Online	5/20/2017	5559	1780539.20	1013784.16	766835.04
2	Personal Care	Offline	5/8/2017	4567	523807.57	363198.03	160609.54
3	Personal Care	Online	3/11/2017	6992	246415.95	170860.05	75555.90
4	Snacks	Online	2/25/2017	7562	1117953.66	713942.88	404010.78
5	Household	Offline	2/8/2017	52284	5997054.98	4509793.96	1487261.02
6	Meat	Online	1/14/2017	8253	2011149.63	1738477.23	272672.40
7	Clothes	Online	1/13/2017	1873	902980.64	296145.92	606834.72
8	Cosmetics	Online	12/31/2016	331438481	3876652.40	2334947.11	1541705.29
9	Office Supplies	Offline	12/6/2016	6213	617347.08	497662.08	119685.00
10	Cosmetics	Offline	11/19/2016	419123971	3039414.40	1830670.16	1208744.24
11	Cosmetics	Online	11/15/2016	286959302	2836990.80	1708748.37	1128242.43
12	Beverages	Offline	10/23/2016	345718562	221117.00	148141.40	72975.60
13	Clothes	Offline	7/25/2016	807025039	600821.44	197048.32	403773.12
14	Snacks	Online	6/30/2016	795490602	339490.50	216804.00	122686.50

Write statements to do the following :

- Display rows 2 to 4 (both inclusive)
- From rows 2 to 4 (both inclusive), display columns, 'Item Type' and 'Total Profit'.
- From rows 2 to 4 (both inclusive), display first four columns.

2:4

SOLUTION

(i) `>>> df[2:5]` `df.iloc[2:5, :]` `df.loc[2:4, :]` `df.loc[2:4]`

In [14]: `df[2:5]`
Out[14]:

	Item Type	Sales Channel	Order Date	Order ID	Total Revenue	Total Cost	Total Profit
2	Personal Care	Offline	5/8/2017	4567	523807.57	363198.03	160609.54
3	Personal Care	Online	3/11/2017	6992	246415.95	170860.05	75555.90
4	Snacks	Online	2/25/2017	7562	1117953.66	713942.88	404010.78

(ii) `>>> df.loc[2:5, ['Item Type', 'Total Profit']]` ★ new

In [18]: `df.loc[2:5, ['Item Type', 'Total Profit']]`
Out[18]:

	Item Type	Total Profit
2	Personal Care	160609.54
3	Personal Care	75555.90
4	Snacks	404010.78
5	Household	1487261.02

(iii) `>>> df.iloc[2:5, 0:4]`

In [19]: `df.iloc[2:5, 0:4]`
Out[19]:

	Item Type	Sales Channel	Order Date	Order ID
2	Personal Care	Offline	5/8/2017	4567
3	Personal Care	Online	3/11/2017	6992
4	Snacks	Online	2/25/2017	7562

Recall that when you use `iloc`, then `<startindex> : <end index>` given for rows and columns work like slices, and the end index is excluded

V) Selecting/Accessing Individual value-

df[colname][rowname]

df.loc[rowname][colname] or **df.loc[rowname,colname]**

df.colname[rowname/row int pos] –TB(1)

Eg- `dtf5.Population['Delhi']`
56789

`dtf5.Population[0]`
56789



	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

df.at[rowname,colname]/loc –TB(2) at-access a **single** value for a **row/column label** pair

df.iat[rowindex,colindex]/iloc –TB(3) iat-access a **single** value for a row/column pair by **integer position** only only only(iat works with integer position)

Eg- `dtf5.at['Chennai','Schools']`
9000

`dtf5.iat[3,2]`
9000

VI) ADDING/MODIFYING ROW'S/COLUMN'S VALUES IN DATAFRAME

- i) Adding/modifying a column
- ii) Adding/modifying a row
- iii) Adding/modifying a single cell

dtf5 →

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

i) Adding/modifying a column-

You can refer to a column in a df in multiple ways

Assigning a value to a column

→ will modify it, if the column already exists

→ will add a new column, if it does not exist already

Syntax- `<df>.<colname>=<new value>`
`<df>['colname']=<new value>`

```
dtf5['Density']=500  
dtf5
```

	Population	Hospitals	Schools	Density
0	56789	190	7000	500
1	80000	300	10000	500
2	75000	150	8000	500
3	46000	350	9000	500

If the colname does not exist in the df, then new column with this name is added.

Prog42- Add a column Density to the dtf5-

Or `dtf5.at[:, 'Density']=500` If this 500 is not there then `at[]` gives an error

Or `dtf5.loc[:, 'Density']=500`

Or `dtf5=dtf5.assign(Density=500)`

Other ways-

Can also be used

temporary

(Density=[500,600,700,200])

Can't add a row or column with `iloc` and `iat`.

Since a column Density does not exist already in the df a new column got added.

*now change the values of the density column-

Prog42-continued

```
dtf5['Density']=[1000,2000,3000,4000]  
#dtf5=dtf5.assign(Density=[1000,2000,3000,4000])  
dtf5
```

If one less value given then error

* note-The sequence which contains the values of the new column must have values equal to number of rows in the df, else python will give an error.

	Population	Hospitals	Schools	Density
0	56789	190	7000	1000
1	80000	300	10000	2000
2	75000	150	8000	3000
3	46000	350	9000	4000

ii) Adding /Modifying a row-

Like columns, you can change or add row to a DF using at or loc attribute as explained-using **at** or **loc**

Syntax- `<df obj>.at[<rowname>,]=<new value>`
`<df obj>.loc[<rowname>,]=<new value>`

If this <new value> is not there then at[] gives an error

If such a row does not exist then python adds a new row else edits its values

Prog43 Add a row Bangalore with value 1200 to dtf5

Note*the new sequence should have values for all the columns, else error
******rows cannot be added using `iloc []` or `iat[]`

```
#prog43  
dtf5.loc['Bangalore']=1200  
dtf5.loc['Bangalore',:]=1200  
dtf5.at['Bangalore']=1200  
dtf5.at['Bangalore',:]=1200  
dtf5
```

	Population	Hospitals	Schools	Density
0	56789	190	7000	1000
1	80000	300	10000	2000
2	75000	150	8000	3000
3	46000	350	9000	4000
Bangalore	1200	1200	1200	1200

```
#eg  
dtf5.loc['Mohali']=[100,200,300]  
dtf5
```

Should be 4 elements in the list

ValueError

Prog-44 pg(55)Eg 36

Chapter 1 : PYTHON PANDAS - I

49

EXAMPLE 36 Consider the following dataframe saleDf :

	Target	Sales
zoneA	56000	58000
zoneB	70000	68000
zoneC	75000	78000
zoneD	60000	61000

```
saleDf['Orders'] = [6000, 6700, 6200, 6000]  
saleDf.loc['ZoneE'] = [50000, 45000, 5000]  
print(saleDf)
```

Write a program to add a column namely *Orders* having values 6000, 6700, 6200 and 6000 respectively for the zones A, B, C and D. The program should also add a new row for a new zone ZoneE. Add some dummy values in this row.

SOLUTION

```
import pandas as pd  
: # saleDf created or loaded here  
saleDf['Orders'] = [6000, 6700, 6200, 6000]  
saleDf.loc['zoneE', :] = [50000, 45000, 5000]  
print(saleDf)
```

Output

	Target	Sales	Orders
zoneA	56000.0	58000.0	6000.0
zoneB	70000.0	68000.0	6700.0
zoneC	75000.0	78000.0	6200.0
zoneD	60000.0	61000.0	6000.0
zoneE	50000.0	45000.0	5000.0

Adding rows from a DataFrame 1.13.2A pg(55,56)

1) Adding rows from a DataFrame having same columns

There is another way of adding rows to a df, especially if the rows to be added are part of another df.

Syntax `df.append()`

Eg1- consider two dfs

df →

	A	B	C
0	1	2	3
1	4	5	6

df1 →

	A	B	C
0	7	8	9
1	10	11	12

→

```
df2=df.append(df1)  
df2
```

	A	B	C
0	1	2	3
1	4	5	6
0	7	8	9
1	10	11	12

The indexes of both the dfs is retained.
But if u want the resultant df to have new indexes
→ then use the parameter **ignore_index=True**



```
import pandas as pd  
df=pd.DataFrame([[1,2,3],[4,5,6]],columns=['A','B','C'])  
print(df)
```

```
df2=df.append(df1,ignore_index=True)  
df2
```

	A	B	C
0	1	2	3
1	4	5	6
2	7	8	9
3	10	11	12

2) Adding rows from a DataFrame having different columns

Eg2→

	A	C
0	1	2
1	4	5

	A	B
0	7	8
1	10	11

```
df2=df.append(df1)  
df2
```

	A	C	B
0	1	2.0	NaN
1	4	5.0	NaN
0	7	NaN	8.0
1	10	NaN	11.0

If True, then raises Value Error on creating index with duplicates

Syntax of the full append ()

`df1.append(<df2>,ignore_index=False,verify_integrity=False,sort=False)`

```
df2=df.append(df1,verify_integrity=True)  
df2
```

ValueError

```
df2=df.append(df1,verify_integrity=True,ignore_index=True)  
df2
```

	A	C	B
0	1	2.0	NaN
1	4	5.0	NaN
2	7	NaN	8.0
3	10	NaN	11.0

Sorts the columns

sort ↓

```
df2=df.append(df1,verify_integrity=True,ignore_index=True,sort=True)  
df2
```

	A	B	C
0	1	NaN	2.0
1	4	NaN	5.0
2	7	8.0	NaN
3	10	11.0	NaN

iii) Modifying a single cell-

You can use any method to access a single cell. Any method which allows you to access a single cell.

Eg- <DF>.iat[rowposition,colposition]=new value
 <DF>.colname[row label/index]=new value

Prog 45- Change the value of population of Bangalore to 5555

```
: #Prog 45
dtf5.Population['Bangalore']=5555
dtf5['Population']['Bangalore']=5555
dtf5.at['Bangalore','Population']=5555
dtf5.loc['Bangalore','Population']=5555

dtf5
```

:

	Population	Hospitals	Schools	Density
0	56789	190	7000	1000
1	80000	300	10000	2000
2	75000	150	8000	3000
3	46000	350	9000	4000
Bangalore	5555	1200	1200	1200

	Population	Hospitals	Schools	Density
0	56789	190	7000	1000
1	80000	300	10000	2000
2	75000	150	8000	3000
3	46000	350	9000	4000
Bangalore	1200	1200	1200	1200

x-----x(Topic)

VII) DELETING/RENAMING COLUMNS/ROWS

Python Pandas gives us 2 ways to delete rows and cols-

- del statement
- drop() function

To rename rows/cols

- rename() function

i) Deleting rows/columns in a DF- Permanent change

(a) Delete a column use del-works with labels

Syntax- `del<df obj>['colname']`

Prog 46-Delete the Density column from dtf5

```
#prog46
del dtf5['Density']
dtf5
```

	Population	Hospitals	Schools
0	56789	190	7000
1	80000	300	10000
2	75000	150	8000
3	46000	350	9000

	Population	Hospitals	Schools	Density
0	56789	190	7000	1000
1	80000	300	10000	2000
2	75000	150	8000	3000
3	46000	350	9000	4000

del	drop
Permanent change	Temporary change
Allows to delete columns	Allows to delete rows and columns
Allows to delete only 1 column at 1 time	Allows to delete 1 or more rows/cols at 1 time

(b) Delete a row use drop()----drop() works with labels

Syntax-

<df>.drop(label or sequence of labels)

Temporary change

	Population	Hospitals	Schools
Delhi	56789	190	7000
Mumbai	80000	300	10000
Kolkata	75000	150	8000
Chennai	46000	350	9000

Prog 47-Delete the rows of Mumbai and Delhi

```
#prog47
dtf5=dtf5.drop(['Delhi','Mumbai'])
dtf5
```

	Population	Hospitals	Schools
Kolkata	75000	150	8000
Chennai	46000	350	9000

To make a permanent change for drop, you can use the inplace argument with drop

	A	B	C
0	15	Arnav	42
1	22	Charles	75
2	35	Guru	66

```
df.drop([0,1],inplace=True)
df
```

	A	B	C
2	35	Guru	66

Or dtf5.drop([0,1])→ this can be used only when the dtf5 has numeric labels of 0 1 2 3.... Else error.

(c)Delete a row/col using drop()-

Syntax-

<df>.drop([label/ sequences of labels],[axis=0/1,inplace=False])

Eg-

```
dtf5=dtf5.drop(['Population','Schools'],axis=1)
dtf5
```

	Hospitals
Delhi	190
Mumbai	300
Kolkata	150
Chennai	350

```
dtf5.drop(['Delhi','kolkata'])
```

default axis=0

	Population	Hospitals	Schools
Mumbai	80000	300	10000
Chennai	46000	350	9000

Prog 48 pg(57)Eg37

EXAMPLE 37 From the `dtf5` used above, create another `DataFrame` and it must not contain the column 'Population' and the row Bangalore.

SOLUTION

```
import pandas as pd
# DataFrame dtf5 created or loaded
dtf6 = pd.DataFrame(dtf5)
del dtf6['Population']
dtf6 = dtf6.drop(['Bangalore'])
print(dtf6)
```

permanent (pointing to `dtf6`)
temporary (pointing to `del dtf6['Population']`)

Output

	Hospitals	Schools
Delhi	189.0	7916.0
Mumbai	208.0	8508.0
Kolkata	149.0	7226.0
Chennai	157.0	7617.0

In this method u can change both at one go!!

iii) Renaming rows/cols labels-

Syntax `<df>.rename(index={change name dictionary},columns={change name dictionary},[inplace=False])`

Or `<df>.rename({change name dictionary},[axis=0/1],[inplace=False])`

If u want to rename row labels then use only index arg

If u want to rename columns labels then use only column arg

If u want to rename both then use both the arguments with dictionaries as {old name:new name}

`inplace`-default False (if `inplace` True then change happens in place and is permanent and None is returned)

Prog- 49 Make the following DF in 3 diff ways and change its row labels to A,B,C,D

	Rollno	Name	Marks
SecA	1	Rishi	97
SecB	2	Arun	98
SecC	3	Rohan	98
SecD	4	Soham	99

```
#prog49 one way
import numpy as np
import pandas as pd
df=pd.DataFrame([[1,'Rishi',97],[2,'Arun',98],[3,'Rohan',98],[4,'Soham',99]],index=['SecA','SecB','SecC','SecD'],columns=['Rollno','Name','Marks'])
df
```

	Rollno	Name	Marks
SecA	1	Rishi	97
SecB	2	Arun	98
SecC	3	Rohan	98
SecD	4	Soham	99

```
#another way
d1={'SecA':1,'SecB':2,'SecC':3,'SecD':4}
d2={'SecA':'Rishi','SecB':'Arun','SecC':'Rohan','SecD':'Soham'}
d3={'SecA':97,'SecB':98,'SecC':98,'SecD':99}
df=pd.DataFrame({'Rollno':d1,'Name':d2,'Marks':d3})
df
```

	Rollno	Name	Marks
SecA	1	Rishi	97
SecB	2	Arun	98
SecC	3	Rohan	98
SecD	4	Soham	99

```
#another way
df=pd.DataFrame({'Rollno':[1,2,3,4],'Name':['Rishi','Arun','Rohan','Soham'],'Marks':[97,98,98,99]},index=['SecA','SecB','SecC','SecD'])
df
```

	Rollno	Name	Marks
SecA	1	Rishi	97
SecB	2	Arun	98
SecC	3	Rohan	98
SecD	4	Soham	99


```
df.rename({'SecA':'A','SecB':'B','SecC':'C','SecD':'D'},inplace=True)
df
```

	Rollno	Name	Marks
A	1	Rishi	97
B	2	Arun	98
C	3	Rohan	98
D	4	Soham	99

	Rollno	Name	Marks
SecA	1	Rishi	97
SecB	2	Arun	98
SecC	3	Rohan	98
SecD	4	Soham	99

Prog50-Write a program to change the column name Rollno to Rno of the following df

```
df.rename({'Rollno':'Rno'},axis=1,inplace=True)
df
```

	Rno	Name	Marks
A	1	Rishi	97
B	2	Arun	98
C	3	Rohan	98
D	4	Soham	99

Prog51 pg(59) Eg38
Prog52 pg(60)Eg39

EXAMPLE 38 Consider the `saleDf` shown below.

	Target	Sales
zoneA	56000	58000
zoneB	70000	68000
zoneC	75000	78000
zoneD	60000	61000

Write a program to rename indexes of 'zoneC' and 'zoneD' as 'Central' and 'Dakshin' respectively and the column names 'Target' and 'Sales' as 'Targeted' and 'Achieved' respectively.

SOLUTION

```
import pandas as pd
: # saleDf created or loaded here
print(saleDf.rename(index = {'zoneC': 'Central', 'zoneD': 'Dakshin'}, \
    columns = {'Target': 'Targeted', 'Sales': 'Achieved'}))
```

Output

	Targeted	Achieved
zoneA	56000	58000
zoneB	70000	68000
Central	75000	78000
Dakshin	60000	61000

EXAMPLE 39 Will the previous program reflect the renamed indexes and columns' names in the dataframe `saleDf`? Make changes in the previous program so that dataframe `saleDf` has the changed indexes and columns.

SOLUTION

The previous program will not reflect the changed indexes and columns' names in the dataframe `saleDf` on which `rename()` is applied because `inplace = True` argument is missing. The changed program to make changes in `saleDf` would be :

```
import pandas as pd
: # saleDf created or loaded here
saleDf.rename( index = {'zoneC': 'Central', 'zoneD': 'Dakshin'}, \
    columns = {'Target': 'Targeted', 'Sales': 'Achieved'}), inplace = True )
print(saleDf)
```

or `saleDf = saleDf.rename(-----`

VIII) Selecting DataFrame Rows/Columns based on Boolean Conditions pg(50)

- Sometimes we need to select rows/cols from a dataframe based on a condition, just the way you filtered the entries in series objects.
- When you compare a dataframe with a value then pandas executes the comparison condition for each element of the df and returns a True/False accordingly for each element.

Prog-53

df →

	A	B	C
0	1	2	3
1	4	5	6
2	7	8	9

→

	A	B	C
0	True	True	True
1	True	True	True
2	True	True	True

When condition is given on the entire df, then it applies the condition on each individual element of the df and returns True and False for each element of the df.

- You can apply condition to individual columns or a range of values too

Prog-54

	people	Amount	Average
0	20	27900	12983.000
1	36	23456	110987.222
2	44	65465	15345.456

df['Average'] > 14000

```
0    False
1     True
2     True
Name: Average, dtype: bool
```

By giving a condition like this, has only given u a result as True or False. But to extract a subset of the df for which the condition is True all u need to do is-

- Write the condition in [] next to the name of the df like-

Syntax-
<df>[condition]
Or
<df>.loc[condition]

df[df['Average'] > 14000]

	people	Amount	Average
1	36	23456	110987.222
2	44	65465	15345.456

df.loc[df['Average'] > 14000]

	people	Amount	Average
1	36	23456	110987.222
2	44	65465	15345.456

Prog-55

	Comm	Sales	Bonus
Soha	12	30	22
Mira	45	33	23
Lina	55	33	50

Display me the details of the employee whose sales are above 30
`df[df['Sales']>30]`

	Comm	Sales	Bonus
Mira	45	33	23
Lina	55	33	50

`df.loc[df['Sales']>30]`

	Comm	Sales	Bonus
Mira	45	33	23
Lina	55	33	50

Internally pandas checks the condition for each row and returns True or False. These truth values act as an index for the rows. The rows with True index are returned.

Eg- SELECTING ROWS AND COLUMNS BASED ON BOOLEAN CONDITIONS-

	name	mks	age	address
0	Ram	100	12	Aundh
1	Sita	99	14	Baner
2	Ravan	78	15	ITRd

#Q1. Display the details of students who stay at baner

```
df[df['address']=='Baner']
```

	name	mks	age	address
1	Sita	99	14	Baner

#Q2. Display the details of students who do not stay at baner

```
df[df['address']!='Baner']
```

	name	mks	age	address
0	Ram	100	12	Aundh
2	Ravan	78	15	ITRd

#Q3. Display the details of students who do not stay at baner and whose age is greater than 14.

```
df[(df['address']!='Baner')&(df['age']>14)]
```

	name	mks	age	address
2	Ravan	78	15	ITRd

	name	mks	age	address
0	Ram	100	12	Aundh
1	Sita	99	14	Baner
2	Ravan	78	15	ITRd

#Q4. Display the details of the students whose marks ≥ 100 or age is greater than or equal to 15.

```
df[(df['mks'] >= 100) | (df['age'] >= 15)]
```

	name	mks	age	address
0	Ram	100	12	Aundh
2	Ravan	78	15	ITRd

#Q4. Display the names and age of the students whose marks ≥ 100 or age is greater than or equal to 15.

```
df[['name', 'age']][(df['mks'] >= 100) | (df['age'] >= 15)]
```

	name	age
0	Ram	12
2	Ravan	15

Creating a New DF from a DataFrame -Shallow vs real copy

Eg→

df

	Comm	Sales	Bonus
Soha	12	30	22
Mira	45	33	23
Lina	55	33	50

```
df1=pd.DataFrame(df)
df1
```

	Comm	Sales	Bonus
Soha	12	30	22
Mira	45	33	23
Lina	55	33	50

```
df['Grade']='A'
df
```

Shallow copy

Here copy=False by default so a shallow copy is made

	Comm	Sales	Bonus	Grade
Soha	12	30	22	A
Mira	45	33	23	A
Lina	55	33	50	A

df1

	Comm	Sales	Bonus	Grade
Soha	12	30	22	A
Mira	45	33	23	A
Lina	55	33	50	A

pg(56)

df1=df.copy()

```
df2=pd.DataFrame(df,copy=True)
df2
```

	Comm	Sales	Bonus
Soha	12	30	22
Mira	45	33	23
Lina	55	33	50

```
df2['Grade']='A'
df2
```

True, Deep copy

	Comm	Sales	Bonus	Grade
Soha	12	30	22	A
Mira	45	33	23	A
Lina	55	33	50	A

df

	Comm	Sales	Bonus
Soha	12	30	22
Mira	45	33	23
Lina	55	33	50

IX) MORE ON DF INDEXING-BOOLEAN INDEXING

Def:- Boolean indexing-means having Boolean values(True or False) or(1 or 0) as indexes of a df.

WHY?

In some cases you may need to divide our data in 2 subsets-True or False

Eg- School decided to have online classes and the schedule may look like

	Day	Classes
True	Mon	6
False	Tue	0
True	Wed	3
False	Thur	0
True	Fri	8

- so we have 2 groups
- 1)True Rows
 - 2)False Rows

This info is useful when we want to find out of when we have online classes and when we don't.

So Boolean indexing divide the df in 2 groups

i) CREATING DF WITH BOOLEAN INDEXING

Prog56- Create the df as above and name it as classdf

```
#Prog52
import pandas as pd
import numpy as np
classdf=pd.DataFrame({'Day':['Mon','Tue','Wed','Thu','Fri'],
                      'Classes':[6,0,3,0,8]},index=[True,False,True,False,True])
classdf
```

	Day	Classes
True	Mon	6
False	Tue	0
True	Wed	3
False	Thu	0
True	Fri	8

Don't put single quotes then it will become string not boolean

```
classdf1=pd.DataFrame({'Day':['Mon','Tue','Wed','Thu','Fri'],
                      'Classes':[6,0,3,0,8]},index=[1,0,1,0,1])
classdf1
```

	Day	Classes
1	Mon	6
0	Tue	0
1	Wed	3
0	Thu	0
1	Fri	8

ii) Accessing rows from df with Boolean indexes-

my doubt↓

We need to make use of

<df>.loc[True]

<df>.loc[False]

<df>.loc[0]

<df>.loc[1]

Prog66- Write the output-

classdf.loc[True]

	Day	Classes
True	Mon	6
True	Wed	3
True	Fri	8

classdf.loc[False]

	Day	Classes
False	Tue	0
False	Thu	0

classdf1.loc[0]

	Day	Classes
0	Tue	0
0	Thu	0

```
df=pd.DataFrame([[1,2,3,4],[6,7,8,9],[1,2,2,1],[3,4,4,4]],index=[1,2,1,2])
df
```

	0	1	2	3
1	1	2	3	4
2	6	7	8	9
1	1	2	2	1
2	3	4	4	4

df.loc[1]

	0	1	2	3
1	1	2	3	4
1	1	2	2	1

df.loc[2]

	0	1	2	3
2	6	7	8	9
2	3	4	4	4

x-----x Python Pandas-1 ends

MCQ'S pg(64,65)solve

Practical Questions-

Q1. Given a series which holds the area of some states in km². Write code to find out the biggest and smallest three areas from the given Series.

```
ds=pd.Series([100,20,30,44,272,65,222])
```

Q2. From the above series find out the areas which are more than 200km².

Q3. Write a Program to create a series object with 6 random integers and having indexes as :['p','q','r','n','t','v']

Q4. Write a program to create data series and then change the indexes of the Series object in any random order.

A1-

```
ds=pd.Series([100,20,30,44,272,65,222])
ds=ds.sort_values()
ds.tail(3)
```

```
0    100
6    222
4    272
dtype: int64
```

A2-

```
ds=pd.Series([100,20,30,44,272,65,222])
ds[ds>200]
```

```
4    272
6    222
dtype: int64
```

A3-

```
ds=pd.Series(np.random.randint(21,size=(6,)),index=['p','q','r','n','t','v'])
ds
```

```
p     9
q     5
r    12
n     8
t     9
v    14
dtype: int32
```

1,21 → for 1 to 20 can be given

A4-

```
ds=pd.Series([100,200,300,400],index=['a','b','c','d'])
print("original series")
print(ds)
ds=ds.reindex(index=['c','a','b','d'])
print("Series after reindexing")
ds
```

```
original series
a    100
b    200
c    300
d    400
dtype: int64
Series after reindexing
c    300
a    100
b    200
d    400
dtype: int64
```

H/W

Q5. WAP to Sort the values of a Series object s1 in ascending order of its values and store it into series object s2

Q6. WAP to Sort the values of a Series object s1 in descending order of its indexes and store it into series object s3

Q7. Given a Series object s4. WAP to change the values at its 2nd row(index1) and 3rd row to 8000

Q8. Given a Series object s5.WAP to calculate the cubes of the Series values.

Q9. Given a Series object s5.WAP to store the squares of the Series values in object s6. Display s6's values which are > 15.

```
#A5
import numpy as np
import pandas as pd
s1=pd.Series([2,1,5,7,6])
s2=s1.sort_values()
print("Series obj s1")
print(s1)
print("Series obj s2")
print(s2)
```

```
Series obj s1
0    2
1    1
2    5
3    7
4    6
dtype: int64
Series obj s2
1    1
0    2
2    5
4    6
3    7
dtype: int64
```

```
#A6
s1=pd.Series([2,1,5,7,6])
s3=s1.sort_index(ascending=False)
print("Series obj s1")
print(s1)
print("Series obj s3")
print(s3)
```

```
Series obj s1
0    2
1    1
2    5
3    7
4    6
dtype: int64
Series obj s3
4    6
3    7
2    5
1    1
0    2
dtype: int64
```

```
#A7
s4=pd.Series([1,2,3,4,5])
print("Original Series obj s4")
print(s4)
s4[1:3]=8000
print("s4 after changing the values")
print(s4)
```

```
Original Series obj s4
0    1
1    2
2    3
3    4
4    5
dtype: int64
s4 after changing the values
0     1
1   8000
2   8000
3     4
4     5
dtype: int64
```

```
#A8
s5=pd.Series([1,2,3,4,5])
print("Series obj s5")
print(s5)
print("Cube of s5 values")
print(s5**3)
```

```
Series obj s5
0    1
1    2
2    3
3    4
4    5
dtype: int64
Cube of s5 values
0     1
1     8
2    27
3    64
4   125
dtype: int64
```

```
#A9
s5=pd.Series([10,2,30])
s6=s5**2
print("values of s6 which are greater than 15")
print(s6[s6>15])
```

```
values of s6 which are greater than 15
0    100
2    900
dtype: int64
```

Q10. WAP to display the number of rows and number of columns in DataFrame df.

Q11. WAP to display the number of rows and number of columns in DataFrame df without the shape attribute.

Q12. Given the df WAP to display the Weight of first and third rows.

df---->

	Age	Name	Weight
0	15	Arnav	42
1	22	Charles	75
2	35	Guru	66

```
df=pd.DataFrame([[10,20,30,40,50],[1,2,3,4,5]])
print("rows",df.shape[0])
print("columns",df.shape[1])
```

```
rows 2
columns 5
```

```
: print("no of rows",len(df.index))
print("no of columns",len(df.columns))
#or
print()
print("no of rows",len(df.axes[0]))
print("no of columns",len(df.axes[1]))
```

```
no of rows 2
no of columns 5
```

```
no of rows 2
no of columns 5
```

```
df.loc[[0,2],'Weight']
#or
df.iloc[[0,2],2]

0    42
2    66
Name: Weight, dtype: int64
```

Q13. Name the data structures of Python's pandas library.

Q14.WAP to create a Series Object Temp1 that stores the temperatures of 7 days in it . Take any random 7 temperatures.

Q15. Make a series same as Q14. and save it in temp2.Index it with 'Mon','Tue'.....

Series
DataFrame
Panel

```
temp1=pd.Series(np.random.randint(30,41,size=(7,)))  
temp1
```

```
0    39  
1    36  
2    33  
3    32  
4    34  
5    40  
6    35  
dtype: int32
```

```
temp2=pd.Series(np.random.randint(30,41,size=(7,)),index=['Mon','Tue','Wed','Thu','Fri','Sat','Sun'])  
temp2
```

```
Mon    39  
Tue    36  
Wed    38  
Thu    40  
Fri    38  
Sat    38  
Sun    35  
dtype: int32
```


Q18. Write a program to create three different series objects from the three columns of a DataFrame df.

Q19. Write a program to create three different series objects from the three rows of a DataFrame df.

```
import numpy as np
import pandas as pd
df=pd.DataFrame([[15,'Arnav',42],[22,'Charles',75],[35,'Guru',66]],columns=['A','B','C'])
print(df)
print("-----")
ds1=df['A']
ds2=df['B']
ds3=df['C']
print(ds1)
print()
print(ds2)
print()
print(ds3)
```

	A	B	C
0	15	Arnav	42
1	22	Charles	75
2	35	Guru	66

0 15
1 22
2 35
Name: A, dtype: int64

0 Arnav
1 Charles
2 Guru
Name: B, dtype: object

0 42
1 75
2 66
Name: C, dtype: int64

```
import numpy as np
import pandas as pd
df=pd.DataFrame([[15,'Arnav',42],[22,'Charles',75],[35,'Guru',66]],columns=['A','B','C'])
print(df)
print("-----")
ds1=df.loc[0]
ds2=df.loc[1]
ds3=df.loc[2]
print(ds1)
print()
print(ds2)
print()
print(ds3)
```

	A	B	C
0	15	Arnav	42
1	22	Charles	75
2	35	Guru	66

A 15
B Arnav
C 42
Name: 0, dtype: object

A 22
B Charles
C 75
Name: 1, dtype: object

A 35
B Guru
C 66
Name: 2, dtype: object

Q20. create a Series from an ndarray which stores characters from 'a' to 'g'

Q21.create a Series that stores the table of number 5

Q22. Write a program to create a df that stores 2 columns which store the series objects of the previous 2 questions (20 and 21)

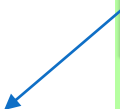
```
arr=np.array([chr(x) for x in range(97,104)])
ds=pd.Series(arr)
print(ds)
```

```
0    a
1    b
2    c
3    d
4    e
5    f
6    g
dtype: object
```

```
#Q-21
i=0
ds=pd.Series([x*5 for x in range(1,11)])
ds
```

```
0     5
1    10
2    15
3    20
4    25
5    30
6    35
7    40
8    45
9    50
dtype: int64
```

Take it as ds1



```
df=pd.DataFrame({'Col1':ds,'Cols2':ds1})
df
```

	Col1	Cols2
0	a	5
1	b	10
2	c	15
3	d	20
4	e	25
5	f	30
6	g	35
7	NaN	40
8	NaN	45
9	NaN	50

Q23- Create a df storing salesmen details(name, zone, sales) of five salesmen.

Q24-Three dictionaries store details of 3 employees as (empno, name). Write a program to create a dataframe from these.

```
df=pd.DataFrame({'Name':['jack','john','smith','leena','Mac'],
                 'zone':['A','B','C','D','E'],
                 'Sales':[100,200,300,100,200]}
                 ,index=['salesman1','salesman2','salesman3','salesman4','salesman5'])
df
```

	Name	zone	Sales
salesman1	jack	A	100
salesman2	john	B	200
salesman3	smith	C	300
salesman4	leena	D	100
salesman5	Mac	E	200

```
d1={'empno':1,'name':'aaa'}
d2={'empno':2,'name':'bbb'}
d3={'empno':3,'name':'ccc'}
df=pd.DataFrame({"One":d1,"Two":d2,"Three":d3})
df
```

or

	One	Two	Three
empno	1	2	3
name	aaa	bbb	ccc

```
d={'empno':1,'ename':'aaa'}
d1={'empno':2,'ename':'bbb'}
list=[d,d1]
df=pd.DataFrame(list)
df
```

	empno	ename
0	1	aaa
1	2	bbb

Even more Dimensions...

After reading the title to this page you might be a little afraid of what's to come. Don't be! Just how we can have lists inside of lists, we can also have dictionaries inside of dictionaries. The important thing to note is that a key in a dictionary cannot be a dictionary itself (**only immutable objects can be dictionary keys**, so things like strings and numbers are okay as keys, but lists and dictionaries are not). However, this does not stop us from having the values of a dictionary be dictionaries. Shown below is an example of how you can make a 2D dictionary:

```
>>> alonzo = {"age": 10, "height": 42, "weight": 175, "instrument ": "fiddle" }  
>>> turing = {"age": 41, "height": 70 "weight": 160, "instrument": "theremin"}  
>>> bertha = {"age": 32, "height": 97, "weight": 587, "instrument": "cello"}  
>>> tinkerB = {"age":100, "height": 4, "weight": 0.5, "instrument": "cello"}  
>>> banditos = {"Alonzo": alonzo, "Turing": turing, "Bertha": bertha, "TinkerB": tinkerB}
```

To return the sub-dictionary representing Alonzo's attributes we do:

```
>>> banditos["Alonzo"]  
{ "age": 10, "height": 42, "weight": 175, "instrument ": "fiddle" }
```

To return a specific trait of Alonzo's we do:

```
>> banditos["Alonzo"]["age"]  
10
```

We can also assign values to the sub-dictionary in a similar way:

```
>>> banditos["Alonzo"]["age"] = 15  
>>> banditos["Alonzo"]["age"]  
15
```

Q25.A list stores 3 dictionaries each storing(old price, new price, change) .wap to create a df from it.

```
d1={'oldprice':1,'newprice':2,'change':1}
d2={'oldprice':5,'newprice':7,'change':2}
d3={'oldprice':2,'newprice':3,'change':3}
list1=[d1,d2,d3]
df=pd.DataFrame(list1)
df
```

	oldprice	newprice	change
0	1	2	1
1	5	7	2
2	2	3	3

Q26. Write code to extract first 10 rows from a dataframe called df using iloc()

df.iloc[0:10,:]

Or

df.iloc[0:10]



Q27-

Questions 13 and 14 make use of this dictionary :

```
my_di = {"name" : ["Jiya", "Tim", "Rohan"], "age" : np.array([10,15,20]), "weight" : (75,123,239), "height" : [4.5, 5, 6.1], "siblings" : 1, "gender" : "M"}
```

13. Predict the output of following code (it uses above given dictionary my_di)

```
df = pd.DataFrame(my_di)
print(df)
```

14. Consider the same dictionary my_di in the previous question, what will be the output following code ?

```
df2 = pd.DataFrame(my_di, index = my_di ["name"] )
print(df2)
```

```
import pandas as pd
import numpy as np
df=pd.DataFrame({'name':'aaa','age':39})
df
```

ValueError

ValueError: If using all scalar values, you must pass an index

↓

```
import pandas as pd
import numpy as np
df=pd.DataFrame({'name':'aaa','age':39},index=[0])
df
```

	name	age
0	aaa	39

A-13

	name	age	weight	height	sibling	gender
0	Jiya	10	75	4.5	1	M
1	Tim	15	123	5.0	1	M
2	Rohan	20	239	6.1	1	M

A-14

	name	age	weight	height	sibling	gender
Jiya	Jiya	10	75	4.5	1	M
Tim	Tim	15	123	5.0	1	M
Rohan	Rohan	20	239	6.1	1	M

Using the `df2` created in Question 14, answer the outputs of questions (15 to 19) given below.

15. Assume that required libraries (panda and numpy) are imported and dataframe `df2` has been created as per questions 13 and 14 above. Predict the output of following code fragment :

```
print( df2["weight"] )
print( df2.weight['Tim' ] )
```

16. Assume that required libraries (panda and numpy) are imported and dataframe `df2` has been created as per questions 13 and 14 above. Predict the output of following code fragment :

```
df2["IQ"] = [130, 105, 115]
df2["Married"] = False
print ( df2 )
```

17. Assume that required libraries (panda and numpy) are imported and dataframe `df2` has been created as per questions 13 and 14 above. Predict the output produced by following code fragment :

```
df2["College"] = pd.Series(["IIT"], index=["Rohan"] )
print ( df2 )
```

18. Assume that required libraries (panda and numpy) are imported and dataframe `df2` has been created as per questions 13 and 14 above. Predict the output produced by following code fragment :

```
print ( df2.loc["Jiya"] )
print ( df2.loc["Jiya", "IQ"] )
print ( df2.loc["Jiya":"Tim" , "IQ":"College"] )
print ( df2.iloc[0] )
print ( df2.iloc[0, 5] )
print ( df2f2.ilocloc[0:2, 5:8] )
```

	name	age	weight	height	sibling	gender
Jiya	Jiya	10	75	4.5	1	M
Tim	Tim	15	123	5.0	1	M
Rohan	Rohan	20	239	6.1	1	M

A-15

```
Jiya      75
Tim       123
Rohan     239
Name: weight, dtype: int64
123
```

A-28

```
print ( df2.loc["Jiya"] )
print ( df2.loc["Jiya","IQ"] )
print ( df2.loc["Jiya":"Tim" , "IQ":"College"] )
print ( df2.iloc[0] )
print ( df2.iloc[0, 5] )
print ( df2.ilocloc[0:2, 5:8] )
```

A-16

	name	age	weight	height	sibling	gender	IQ	Married
Jiya	Jiya	10	75	4.5	1	M	130	False
Tim	Tim	15	123	5.0	1	M	105	False
Rohan	Rohan	20	239	6.1	1	M	115	False

A-18

df2.loc["Jiya"]

```
name      Jiya
age       10
weight    75
height    4.5
sibling    1
gender     M
IQ       130
Married   False
college   NaN
Name: Jiya, dtype: object
```

df2.iloc[0]

```
name      Jiya
age       10
weight    75
height    4.5
sibling    1
gender     M
IQ       130
Married   False
college   NaN
Name: Jiya, dtype: object
```

df2.iloc[0,5]

'M'

df2.iloc[0:2,5:8]

	gender	IQ	Married
Jiya	M	130	False
Tim	M	105	False

A-17

```
df2['college']=pd.Series(['IIT'],index=['Rohan'])
df2
```

	name	age	weight	height	sibling	gender	IQ	Married	college
Jiya	Jiya	10	75	4.5	1	M	130	False	NaN
Tim	Tim	15	123	5.0	1	M	105	False	NaN
Rohan	Rohan	20	239	6.1	1	M	115	False	IIT

	name	age	weight	height	sibling	gender
Jiya	Jiya	10	75	4.5	1	M
Tim	Tim	15	123	5.0	1	M
Rohan	Rohan	20	239	6.1	1	M

Q29- write the output of the following-

```
d={'one':pd.Series([1.,2.,3.],index=['a','b','c']),'two':pd.Series([1.,2.,3.,4.],index=['a','b','c','d'])}  
df=pd.DataFrame(d)  
df1=pd.DataFrame(d,index=['d','b','a'])  
df2=pd.DataFrame(d,index=['d','a'],columns=['two','three'])  
print(df)  
print(df1)  
print(df2)
```

Ans-29

	one	two
a	1.0	1.0
b	2.0	2.0
c	3.0	3.0
d	NaN	4.0
	one	two
d	NaN	4.0
b	2.0	2.0
a	1.0	1.0
	two	three
d	4.0	NaN
a	1.0	NaN

Q30.From the earlier df display:-

1)only row 'a' from df,df1,df2

```
print(df.loc['a'],df1.loc['a'],df2.loc['a'])  
  
one      1.0  
two      1.0  
Name: a, dtype: float64 one      1.0  
two      1.0  
Name: a, dtype: float64 two      1  
three    NaN  
Name: a, dtype: object
```

2)add an empty columns 'x' to all the dfs.

3)display rows 0 and 1 from the three dfs

```
df['x']=""  
df1['x']=""  
df2['x']=""  
print(df,df1,df2)  
  
      one  two x  
a  1.0  1.0  
b  2.0  2.0  
c  3.0  3.0  
d  NaN  4.0      one  two x  
d  NaN  4.0  
b  2.0  2.0  
a  1.0  1.0      two three x  
d  4.0  NaN  
a  1.0  NaN
```

Empty gives false

```
df.iloc[0:2]
```

	one	two	x	y
a	1.0	1.0		NaN
b	2.0	2.0		NaN